Rethinking Modern Forward+ Shading

Dual-HiZ Adaptive Clustered Forward+ Architecture

Péter Tamás, András Máté Fridvalszky

2026, Budapest Graphics Forum

Introduction

About me:

- 23 years old
- **M.Sc. Student** in Computer Engineering and Teaching at BME
- Planning to **continue with a Ph.D.** in the field of computer graphics.

Research interests:

- High-Performance Computing
- **GPU-Driven Rendering Architectures** ☺☺☺
- Modern Game Engine Design

Current Work:

- **Developer of the Synapse Engine:** A modern, research and education oriented GPU-driven game engine.



Table of Content

Section I: Overview of **Current Shading Techniques**

1. Forward
2. Deferred
3. Forward+

Section II: Deep Dive into the **Dual-Hiz Adaptive Clustered Forward+**

- Architecture
- Shader Codes
- Benchmarks



Forward Shading

How it works:

1. Rasterize geometry
2. Shade fragments immediately.
3. Iterates over **all Directional, Point, and Spot lights**.
4. Accumulate light contributions.

Advantages:

- Simplest Rendering Approach
- Supports **Complex Materials**
- Handles **Transparent** and Opaqe object simultaneously



```
void main()
{
    vec3 finalColor = vec3(0.0);

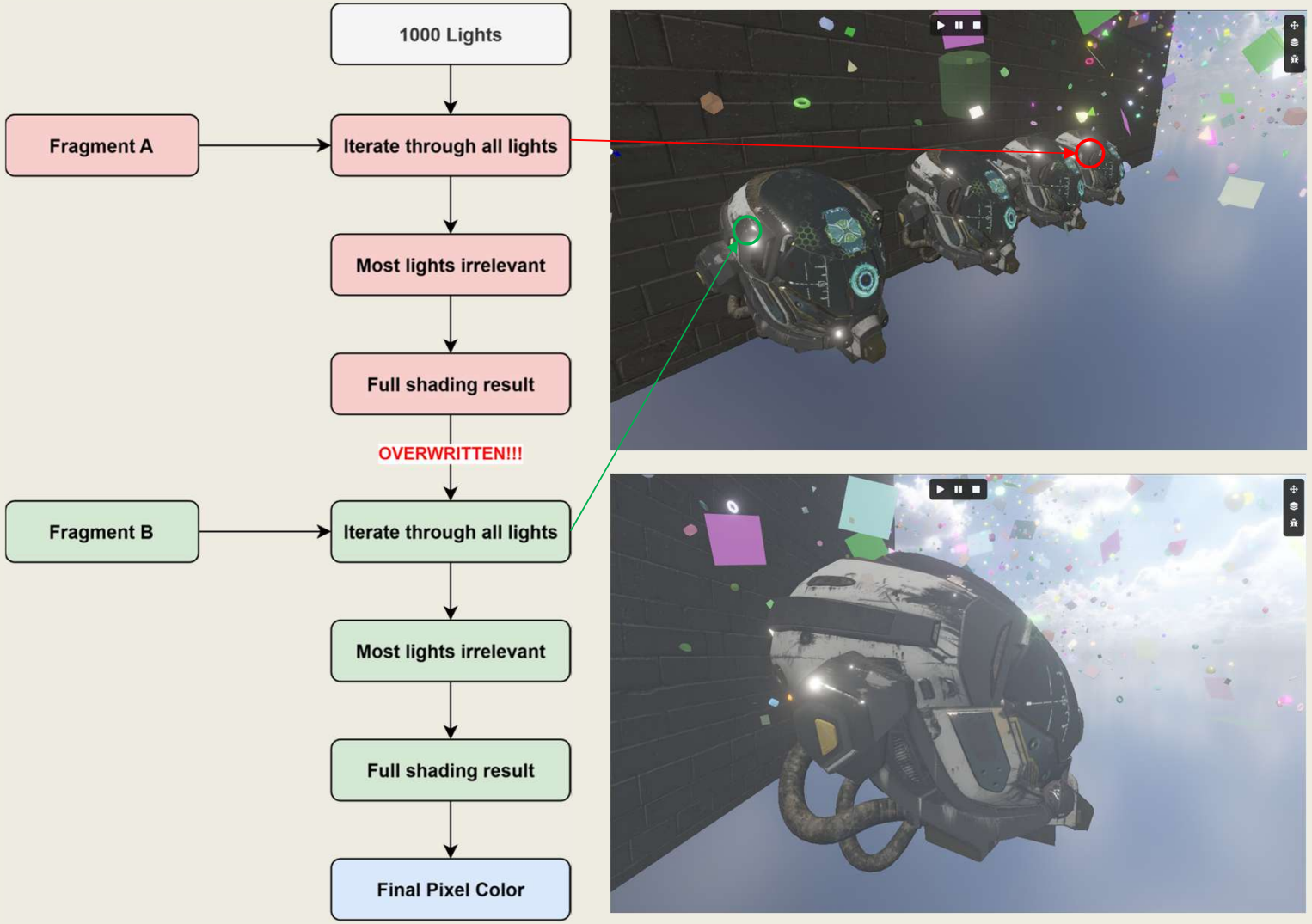
    for (uint i = 0; i < directionalLightCount; ++i)
        finalColor += CalculateDirectionalLight(...);

    for (uint i = 0; i < pointLightCount; ++i)
        finalColor += CalculatePointLight(...);

    for (uint i = 0; i < spotLightCount; ++i)
        finalColor += CalculateSpotLight(...);

    outputColor = vec4(finalColor, 1.0);
}
```

Forward Shading: The Main Problems



Deferred Shading I.

Two-Pass Rendering:

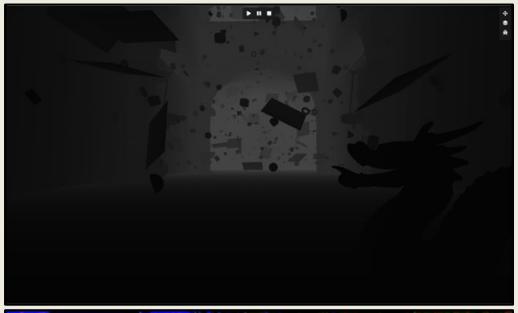
1. G-Buffer Pass
2. Lighting Pass

What G-Buffer Solves:

- Solves **lighting overdraw**
- Shades only final visible geometrie's fragments.
- Reduces **wasted GPU work**



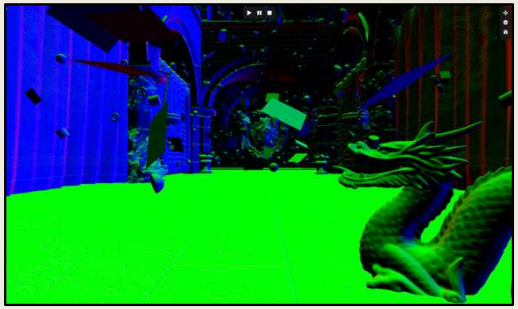
← Final



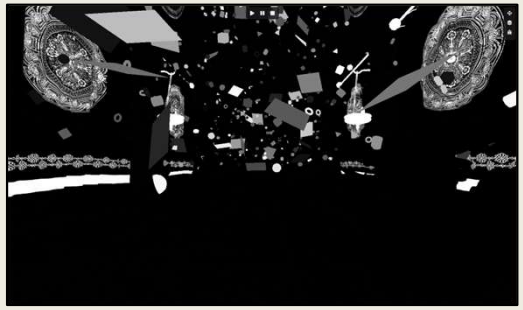
Depth →



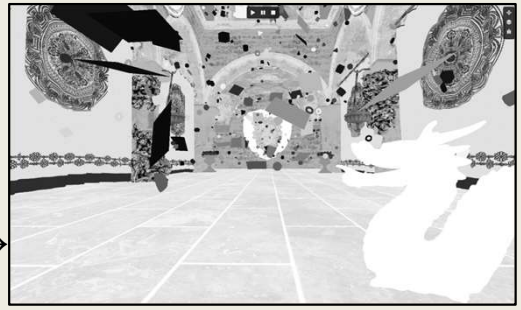
← Color



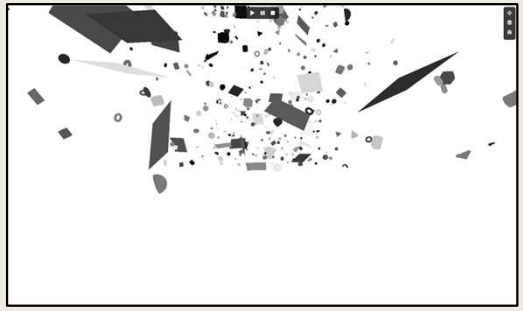
Normal →



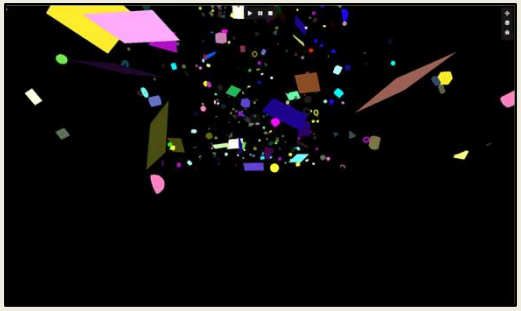
← Metallic



Roughness →



← AO



Emissive →

Deferred Shading II.

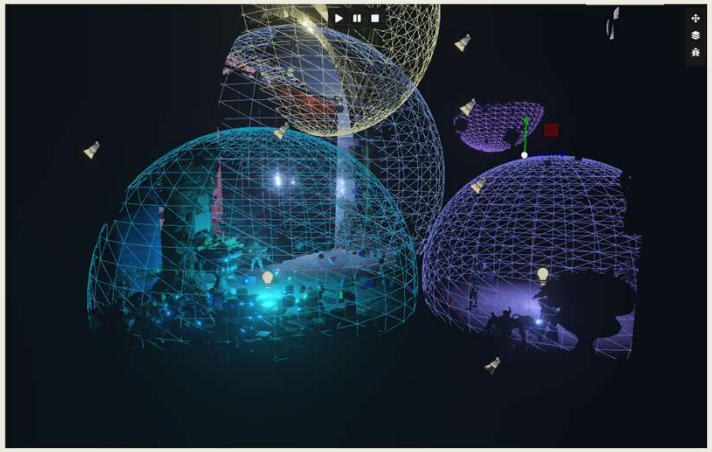
Light Volume Rendering

- Render **Proxy Volumes!!**
- Point → Cube proxy
- Spot → Pyramid proxy

Advantages:

- Proxy volumes are **cheaper** to render
- Shades **only affected** screen pixel regions!!!
- **Cull non-visible light** volumes before shading

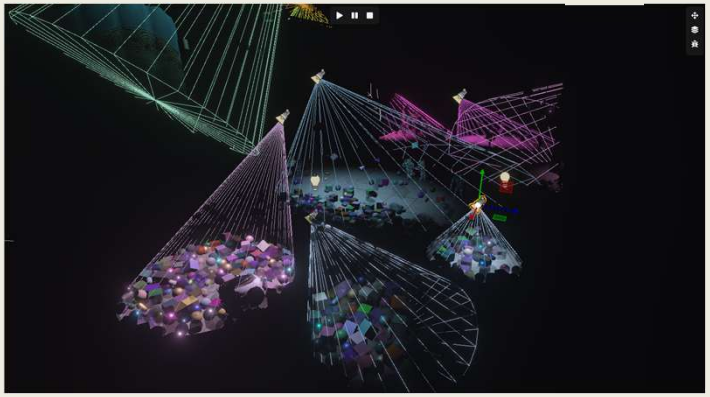
Point Light Sphere Volume ⬇



Point Light Proxy Cube ⬇



Spot Light Cone Volume ⬇



Spot Light Proxy Pyramid ⬇



Deferred Shading III.

Additive Light Accum.

1. Start from **black image**
2. Render lights separately
3. Read G-Buffer
4. Add light contribution using **additive blending**

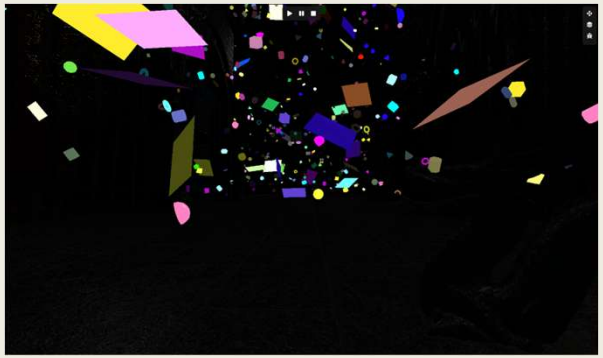
Problems:

- High G-Buffer **memory usage**
- Complex materials need larger G-Buffers…
- No support for **Transparency**

Main Problems:

- **Repeated** reads of G-Buffer…
- **High memory bandwidth** cost

Emissive/ Ambient Color ↓



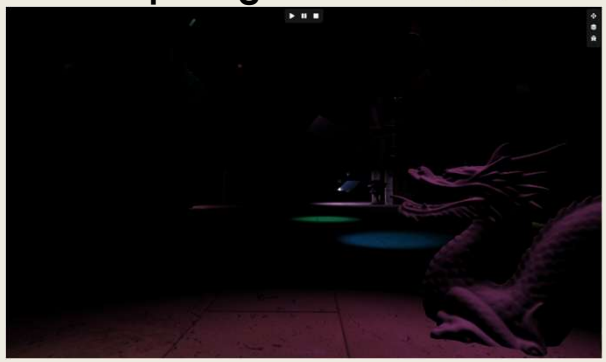
Direction Light Simulation ↓



Point Light Simulation ↓



Spot Light Simulation ↓



Additive Light Simulation ↓



Bloom Simulation ↓



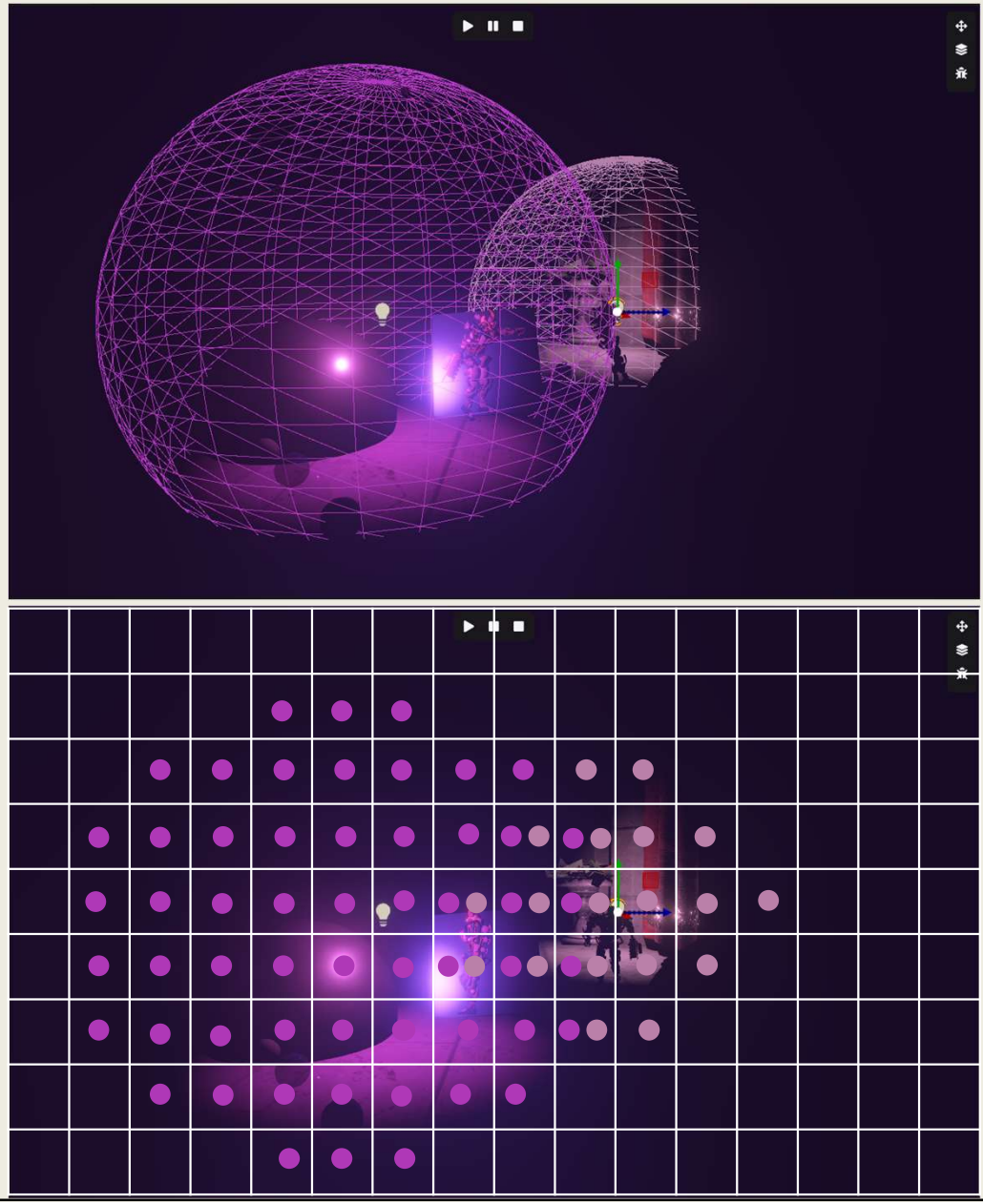
Tiled Forward+ Shading

How it works:

- Return to Forward Shading
- Screen-space partitioning:
  1. Divide the **screen into 2D tiles**
  2. Store **intersecting lights** for each tile
  3. Shade only with the **tile's light list**

Problems:

- Partitioning **only in X/Y**
- No depth partitioning…
- Many **false-positive lights** remain ☹



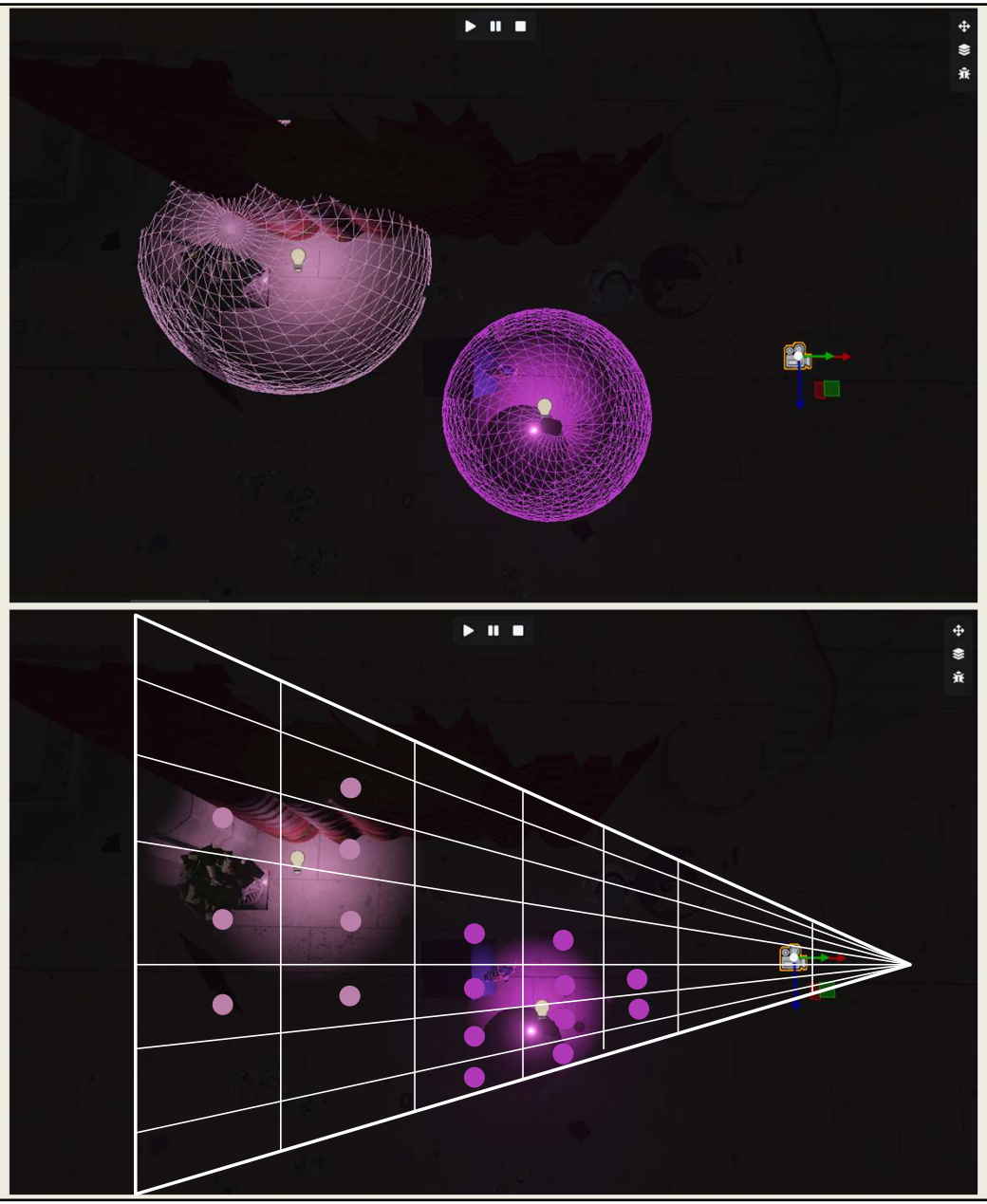
Clustered Forward+ Shading I.

How it works:

- Extension of Tiled Forward
- World-space partitioning:
  1. Partition the camera frustum in **X/Y/Z**
  2. Creates a **3D cluster grid**
  3. Assign lights to intersecting clusters

Examples:

- **DOOM (2016)**



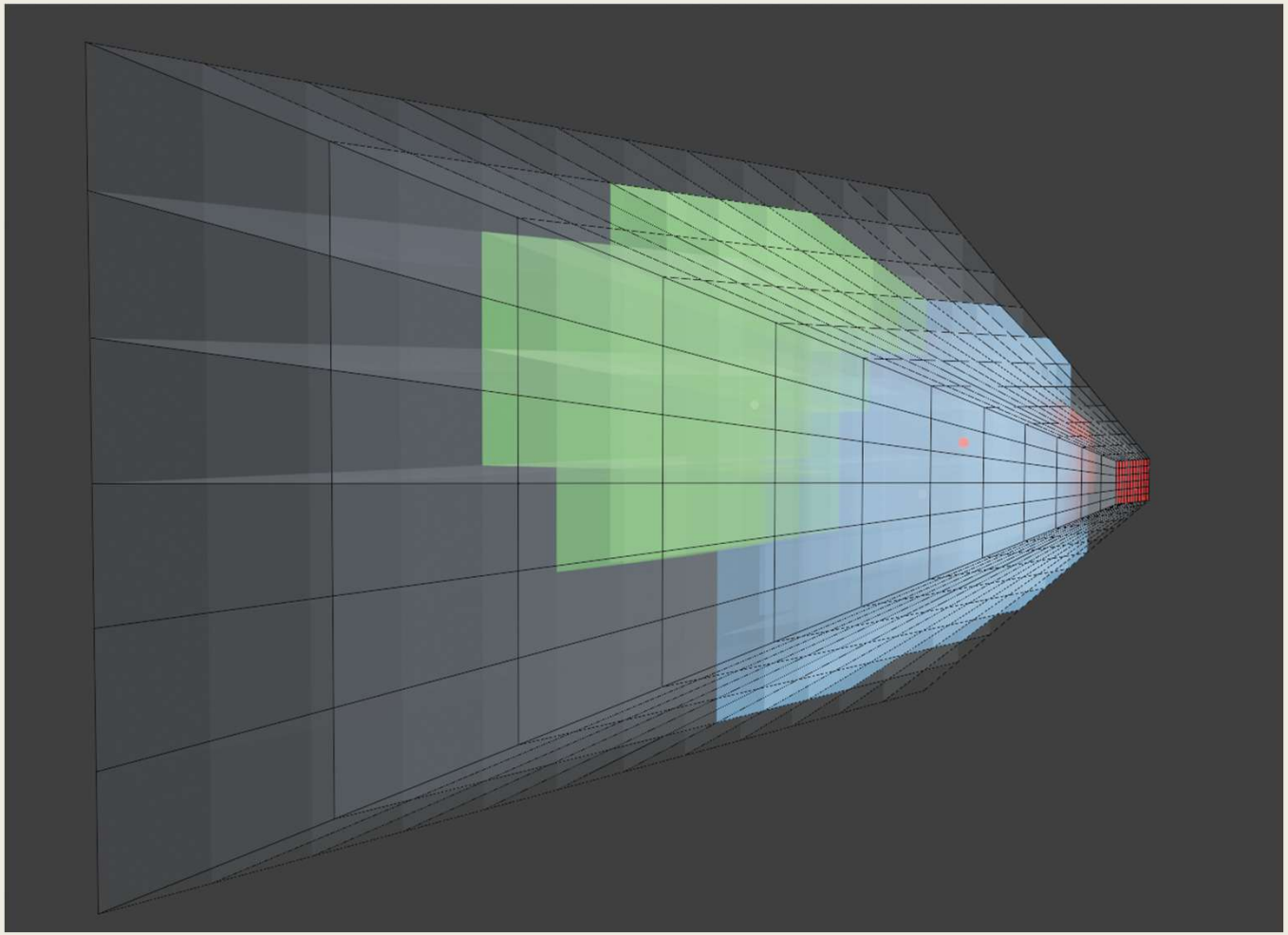
Clustered Forward+ Shading II.

Advantages:

- Better spatial partitioning
- Each fragment evaluates only its **cluster's light list**
- **Handles Transparent** and Opaque simultaneously!!

Problems:

- Partitions the **entire camera frustum**
- Creates many **empty clusters**
- Empty clusters still increase **culling overhead**



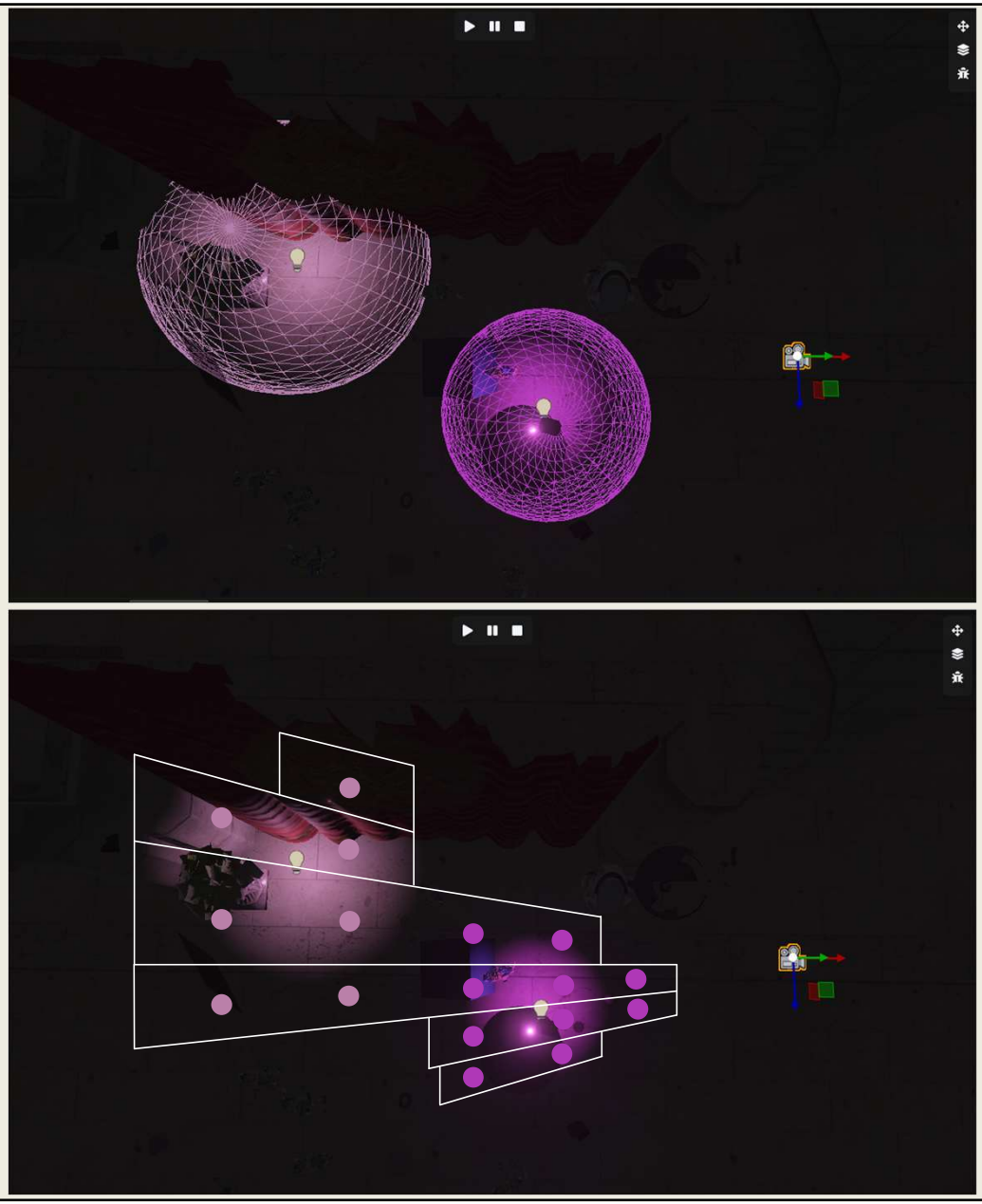
Depth-Bounded Forward+ Shading I.

How it works:

- **Visible geometry defines the depth volume!!**
- Create 3D clusters only inside this range
- Cull lights inside the reduced frustum
- **One compute pass** handles the complete workflow…

Examples:

- **Harada:** 2.5D Culling (2012)
- **Wicked Engine:** Bitmasked Harada Culling



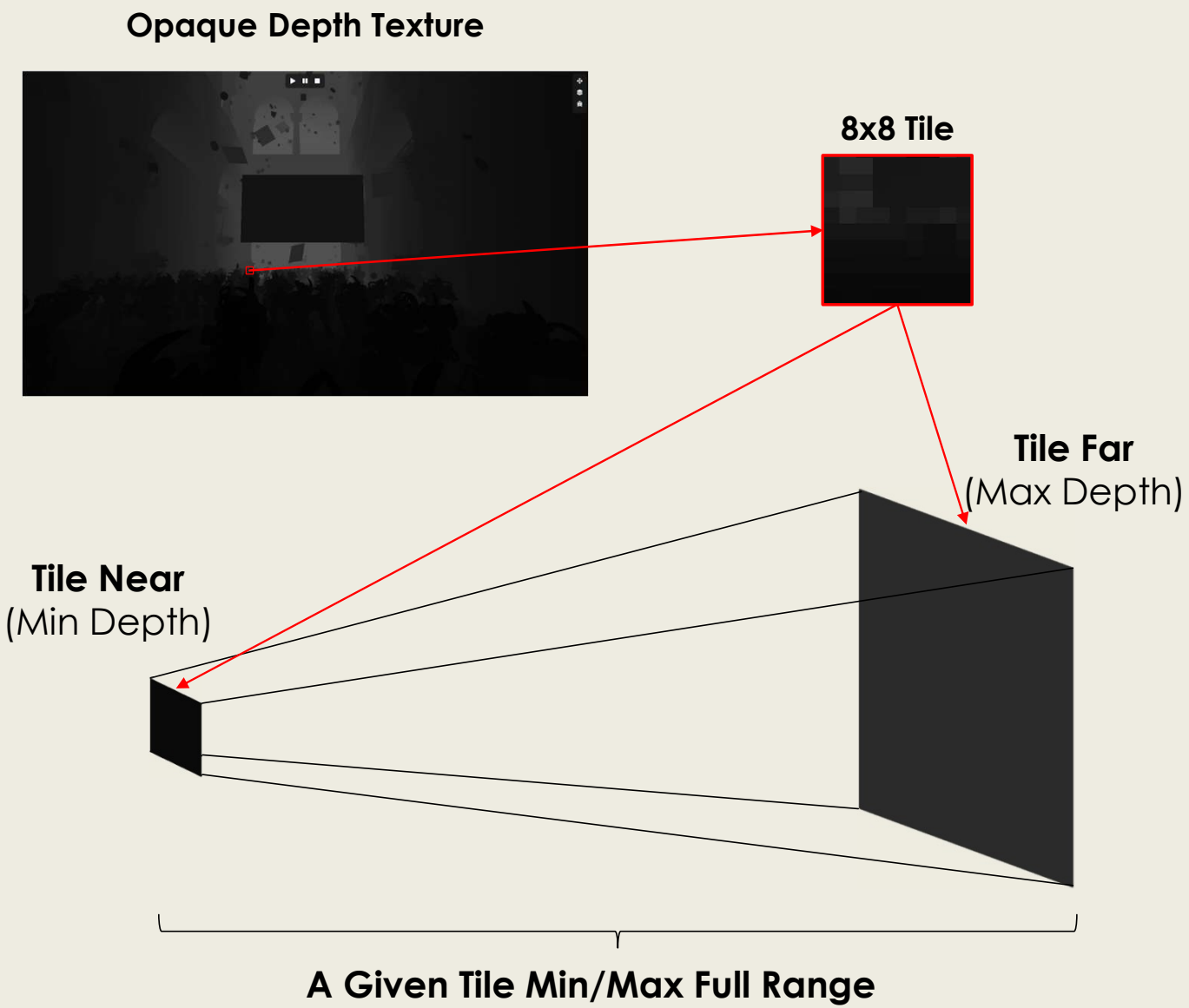
Depth-Bounded Forward+ Shading II.

Tile Processing:

1. Read 8x8 or 16x16 tile depth texture values
2. Compute min/max depth
3. Build one **depth-bounded frustum per tile**

Problems:

- One tile has **one large depth range**
- Large cluster causes lots of false positives still
- **Transparent geometry** needs separate handling!!
- One large uber shader ☹ ☹ ☹



Depth-Bounded Forward+ Shading III.

Light volumes are solved by spatial partitioning. **But what about overdraw?**

Solution: Depth-Prepass:

1. Render the full scene **DEPTH_ONLY**
2. Use the depth buffer to build **LIGHT LIST**
3. Render the scene again with **DEPTH_EQUAL**

Result:

- Shade only the **final visible fragments.**
- Eliminates **lighting overdraw!!**

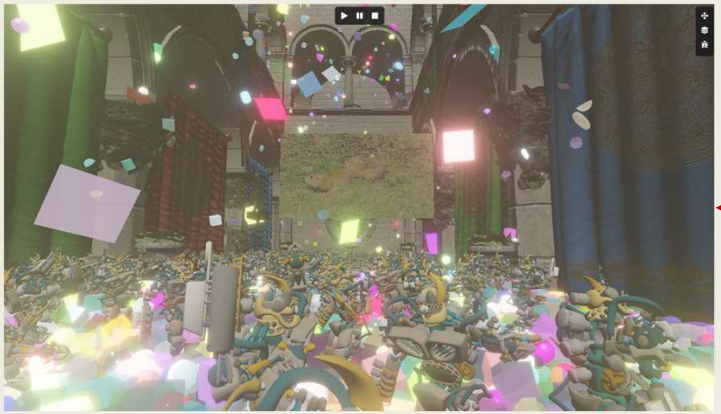
1. Depth Prepass



2. Cluster Setup and Light Culling



3. Final scene with DEPTH_EQUALS



Forward+ Techniques: Limitations/Problems

1. Clustered Forward+: wastes GPU work on **empty clusters**.
2. Depth-Bounded Forward+: use only **one depth range per tile**.
3. Depth-Bounded Forward+: **struggles to handle transparent** geometry lighting efficiently.
4. **Fixed light list allocation** limits lights per cluster.
5. **Large tiles are difficult** to handle efficiently. (64x64/128x128/256x256).
6. Fixed small tiles (8x8, 16x16) **scale poorly at 4K and 8K** resolutions.
7. Many **designs** target **older GPU** and **graphics API** architectures.

Dual-HiZ Adaptive Clustered Forward Shading

An Original Rendering Technique by Péter Tamás

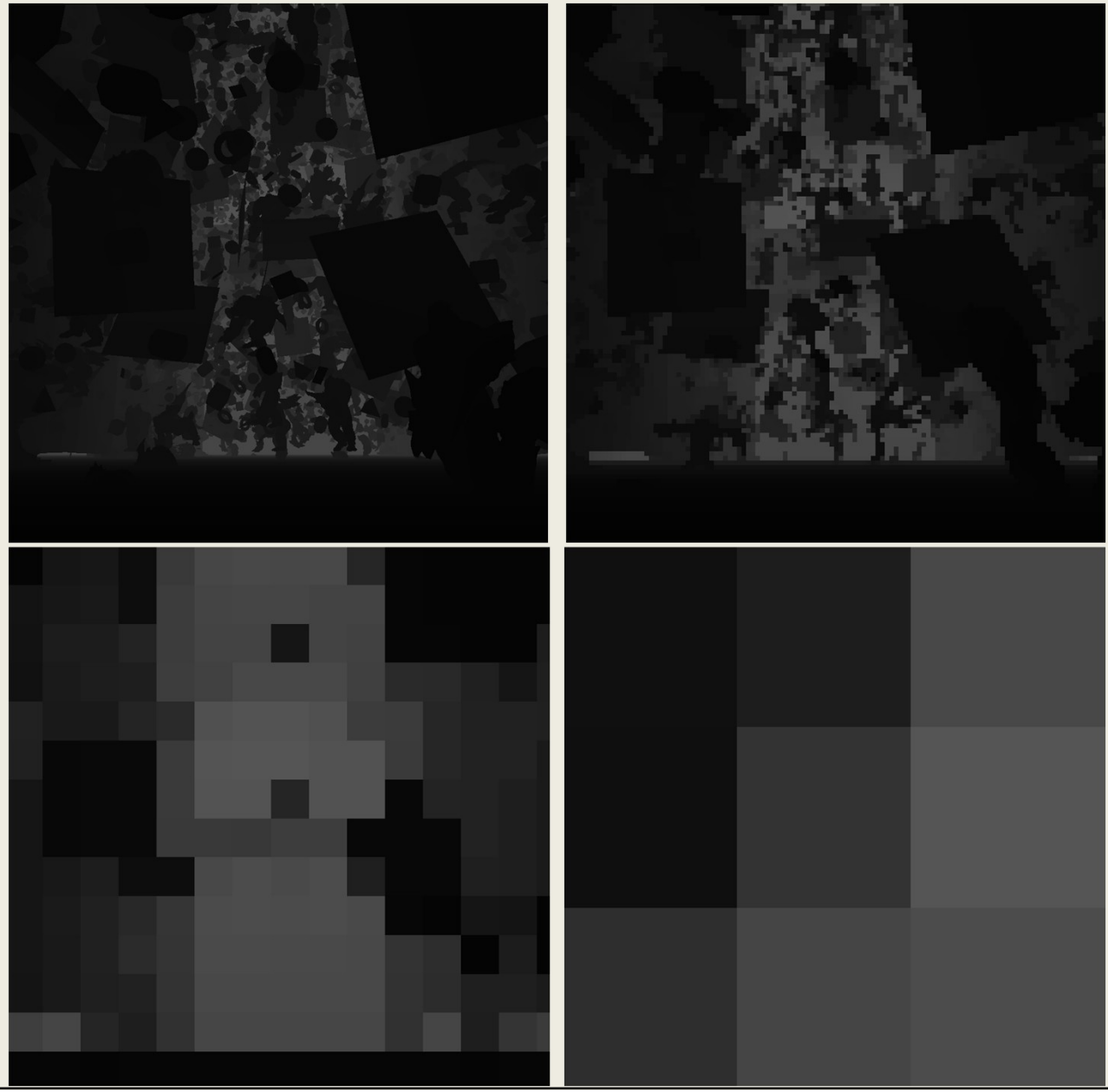
Dual-HiZ Adaptive Clustered

Origin of Single-HiZ Architecture:

- **GPU-Driven Hi-Z Occlusion Culling!!!**
- **Max-Reduction Hi-Z Pyramid** built on opaque depth
- Build Cost: **~0.08 ms**
- **AMD FidelityFX** Single Pass Downsampler even faster!!
- **Mip Screen-Space Regions**
  … → 16×16 → 32×32 → 64×64 →128×128 → 256×256 → …

Single-HiZ Idea:

- Tile Max Depth = Frustum Far Plane
- Only **ONE Texture Read** per Tile!!
- Works with **Any Tile Size**!!



Dual-HiZ Adaptive Clustered

Dual-HiZ Architecture:

- **Opaque Depth:**
  - → Max Hi-Z
  - → Furthest Opaque
- **Opaque + Transparent Depth:**
  - → Min Hi-Z
  - → Closest Visible Surface

Tile Frustum:

- Near Plane = Min Hi-Z
- Far Plane = Max Hi-Z

Result:

- **Correct for Opaque and Transparent Objects!!!**
- No Per-Pixel Depth Reduction Needed!!!







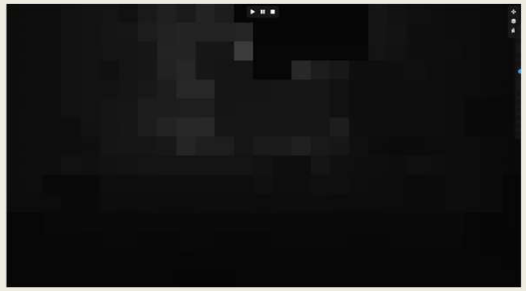
Mip0:
1x1 Tile





Mip4:
16x16 Tile





Mip6:
64x64 Tile

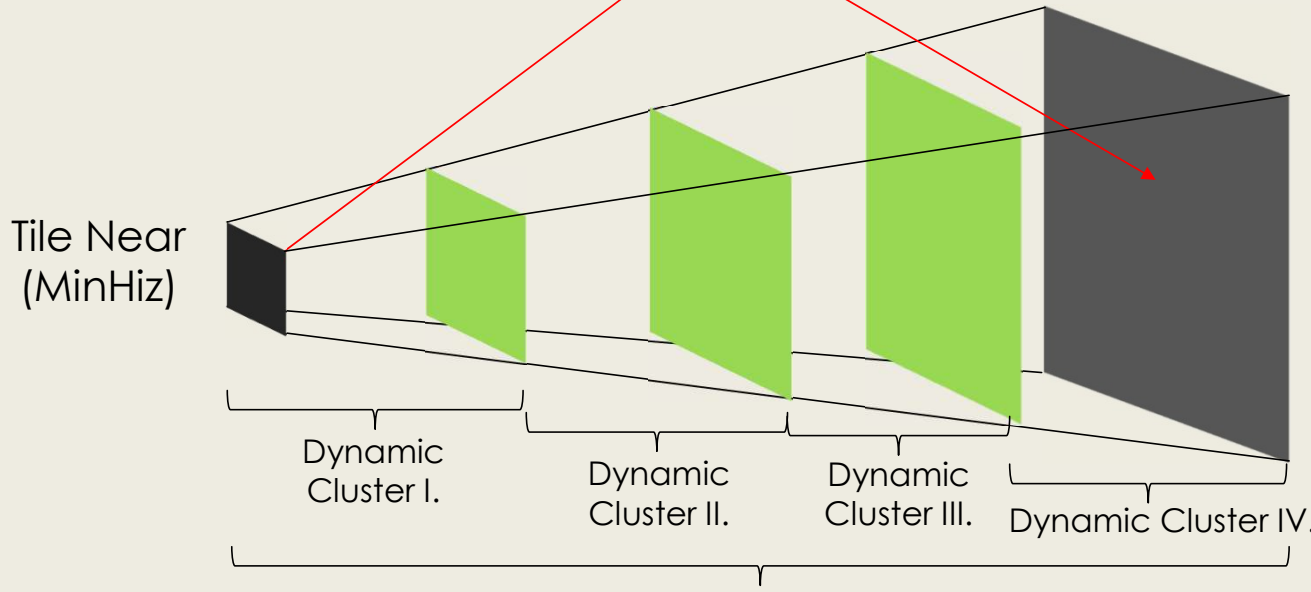
Dual-Hiz Adaptive Clustering

Opaque
Max HiZ
Mip6: 64x64 Tile





Opaque + Transparent
Min HiZ
Mip6: 64x64 Tile



A Given Tile HiZ Min/Max Full Range

1. Part of the Shader

16 Threads / Tile (Max 16 Cluster)

```
layout(local_size_x = 16, local_size_y = 1, local_size_z = 1) in;
layout(set = 2, binding = 0) uniform sampler2D hizDepth;

shared float s_minZ;
shared float s_maxZ;
shared uint s_sliceCount;

void main() {
    FrameGlobalContext ctx = GET_FRAME_CONTEXT(pc.frameGlobalContextBufferAddr);

    uint threadId = gl_LocalInvocationID.x;
    ivec2 tileId = ivec2(gl_WorkGroupID.xy);
    uint flattenTileIdx = tileId.y * ctx.tileCountX + tileId.x;

    if (threadId == 0) {
        uint cameraIdx = GET_SPARSE_INDEX(ctx.cameraSparseMapBufferAddr, ctx.mainCameraEntity);
        CameraComponent camera = GET_CAMERA(ctx.cameraBufferAddr, cameraIdx);
        float near = GET_CAMERA_NEAR(camera);
        float far  = GET_CAMERA_FAR(camera);
```

HiZ Texture Read (Tile Near/Far Planes)

```
        ivec2 mipSize = textureSize(hizDepth, int(ctx.hizMipLevel));
        ivec2 texelCoord = min(tileId, mipSize - 1);
        vec2 hizValue = texelFetch(hizDepth, texelCoord, int(ctx.hizMipLevel)).xy;

        float zMax = near + hizValue.x * (far - near);
        float zMin = near + hizValue.y * (far - near);

        //Background -> Skip
        if (hizValue.y >= 0.9999) {
            s_sliceCount = 0;
        }
```

Slice Up Cluster Adaptively

```
        else {
            s_minZ = max(zMin, near);
            s_maxZ = min(zMax, far);
            s_sliceCount = CalculateClusterSliceCount(s_minZ, s_maxZ, ctx.sliceScaleFactor);
        }
    }
    barrier();
    //... rest of the code
}
```

2. Part of the Shader

```
void main(){
    //... Previous code
```

Subgroup Based Atomic Cluster Registration!!

```
    uint needsWrite = (threadId < s_sliceCount) ? 1 : 0;
    uint subgroupWriteCount = subgroupAdd(needsWrite);
    uint localOffset = subgroupExclusiveAdd(needsWrite);
    uint baseOffset = 0;

    if (subgroupElect() && subgroupWriteCount > 0) {
        baseOffset = atomicAdd(GET_CLUSTER_COUNT(ctx.forwardPlusClusterCountBufferAddr), subgroupWriteCount);
    }

    baseOffset = subgroupBroadcastFirst(baseOffset);
```

Register Tile Descriptor

```
    if (threadId == 0) {
        TileData tileData;
        tileData.minZ = s_minZ;
        tileData.maxZ = s_maxZ;
        tileData.sliceCount = s_sliceCount;
        tileData.clusterBaseOffset = baseOffset;
        GET_TILE_DATA(ctx.forwardPlusTileGridListBufferAddr, flattenTileIdx) = tileData;
    }
```

Sliced Cluster Near and Far

```
    if (needsWrite == 1) {
        float sliceMinZ, sliceMaxZ;

        if (s_sliceCount == 1) {
            sliceMinZ = s_minZ;
            sliceMaxZ = s_maxZ;
        }
        else {
            float ratio = s_maxZ / s_minZ;
            sliceMinZ = s_minZ * pow(ratio, float(threadId) / float(s_sliceCount));
            sliceMaxZ = s_minZ * pow(ratio, float(threadId + 1) / float(s_sliceCount));
        }
```

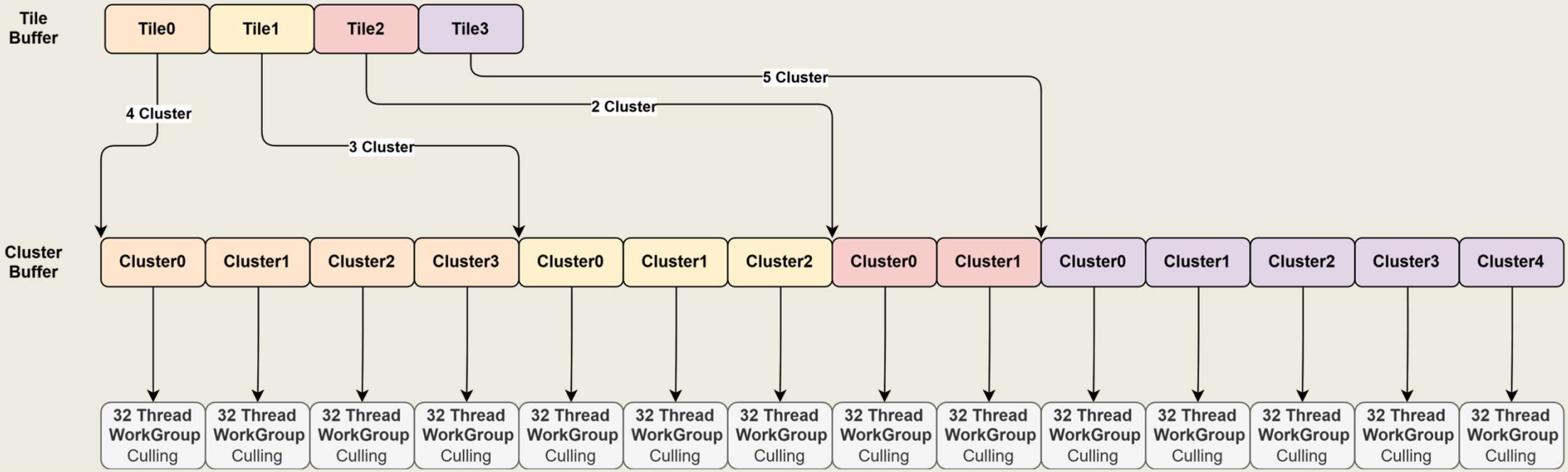
Register Cluster Descriptor (Atomic Index)

```
        uint finalIndex = baseOffset + localOffset;

        ClusterData clusterData;
        clusterData.tileIndex = flattenTileIdx;
        clusterData.sliceIndex = threadId;
        clusterData.minZ = sliceMinZ;
        clusterData.maxZ = sliceMaxZ;
        clusterData.pointLightCount = 0;
        clusterData.pointLightOffset = 0;
        clusterData.spotLightCount = 0;
        clusterData.spotLightOffset = 0;

        GET_CLUSTER_DATA(ctx.forwardPlusClusterListBufferAddr, finalIndex) = clusterData;
    }
}
```

Tile/Cluster: Memory Layout



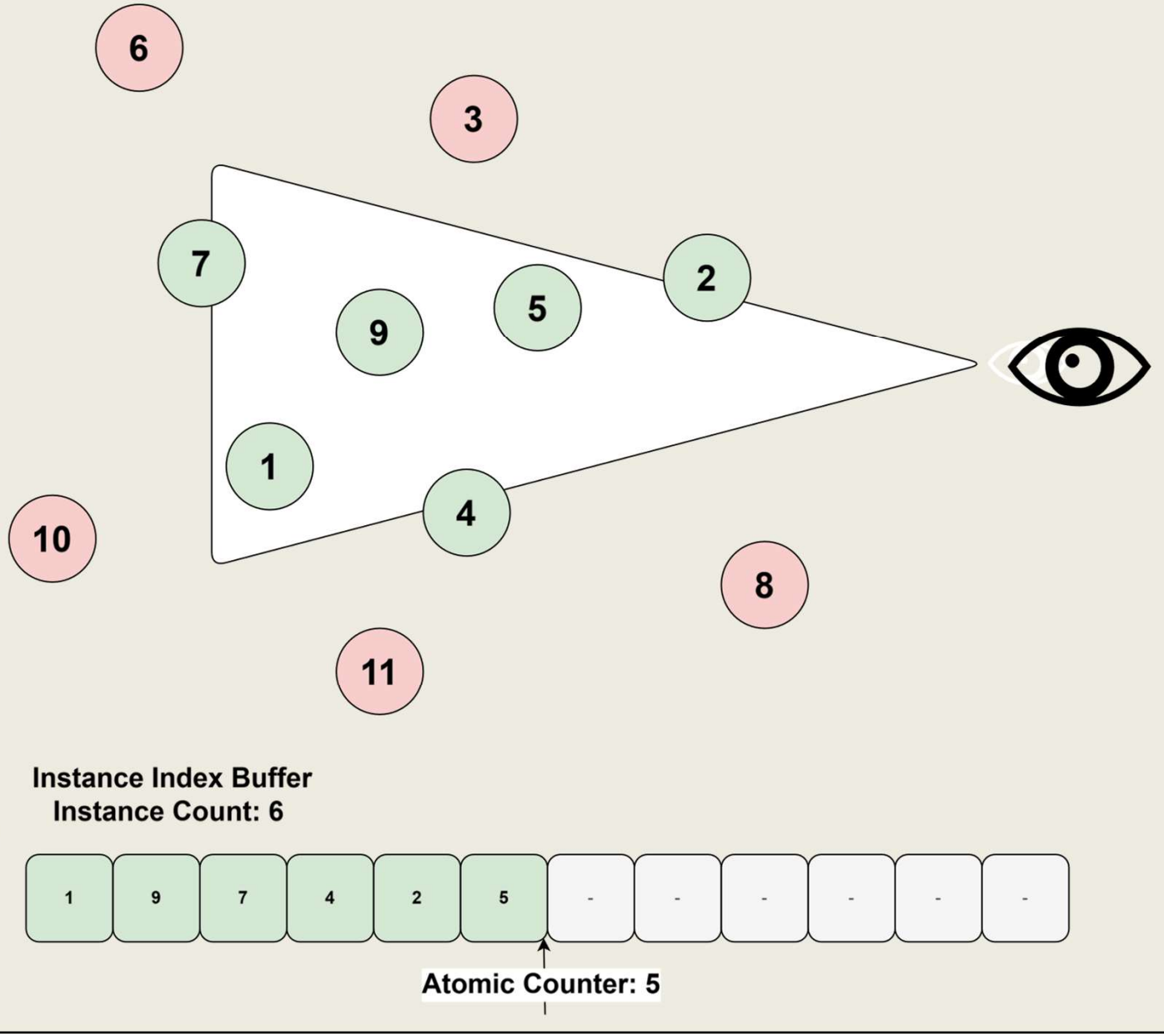
Dual-HiZ Adaptive Clustered

Two-Stage Light Culling on GPU:

- **Camera Frustum vs Lights:**
  - → Cull Point + Spot Lights
  - → Keep only visible lights!
- **Clusters vs Visible Lights:**
  - → Test clusters only against the already visible light list

Result:

- No cluster tests against invisible scene lights!!



Dual-HiZ Adaptive Clustered

Collaborative Light Culling:

- **32 Threads** per Workgroup
- **Point Light** Culling Shader
- **Spot Light** Culling Shader
- Separate Visible Light Index Lists!!

Point Light Culling:

- Cluster **AABB** vs Point **Sphere** Intersection
- Cheap Intersection Test

Map Threads IDs
32 Thread/Cluster

Cluster vs Point
Intersection Test

Subgroup Optimised
Intersection Count
+ Atomic Add!!

Visible Light Index
Registration

```
layout(local_size_x = 32, local_size_y = 1, local_size_z = 1) in;

void main()
{
    uint threadId = gl_LocalInvocationID.x;
    uint clusterIdx = gl_WorkGroupID.x;

    // 1. Retrive context, camera, cluster data
    // 2. Calculate cluster AABB in view space
    // 3. Retrive total lights and global offset for the cluster

    barrier();

    for (uint i = threadId; i < totalLights; i += gl_WorkGroupSize.x) {
        uint entityIdx = GET_POINT_VISIBLE_LIGHT(ctx.pointLightVisibleIndexBufferAddr, i);
        uint denseIdx = GET_SPARSE_INDEX(ctx.pointLightSparseMapBufferAddr, entityIdx);
        PointLightColliderGPU collider = GET_POINT_LIGHT_COLLIDER(ctx.pointLightColliderBufferAddr, denseIdx);

        vec3 viewSpaceCenter = (camera.view * vec4(collider.center, 1.0)).xyz;
        bool isIntersecting = TestSphereAABB(viewSpaceCenter, collider.radius, s_clusterAABBMin, s_clusterAABBMax);

        uvec4 ballot = subgroupBallot(isIntersecting);
        uint activeCount = subgroupBallotBitCount(ballot);

        uint baseOffset = 0;
        if (subgroupElect() && activeCount > 0) {
            baseOffset = atomicAdd(s_localPointLightCount, activeCount);
        }
        baseOffset = subgroupBroadcastFirst(baseOffset);

        uint isIntersectingInt = isIntersecting ? 1 : 0;
        uint threadOffset = subgroupExclusiveAdd(isIntersectingInt);

        if (isIntersecting) {
            uint localIdx = baseOffset + threadOffset;

            if (localIdx < MAX_LIGHTS_PER_CLUSTER) {
                uint finalIdx = globalOffset + localIdx;
                GET_LIGHT_INDEX(ctx.forwardPlusPointLightIndexListBufferAddr, finalIdx) = entityIdx;
            }
        }
    }

    barrier();

    // 4. Write the point light count and offset to the cluster data
}
```

Dual-HiZ Adaptive Clustered

Problem with Spot Lights:

- Spot Light **Sphere/AABB** collider is **too large**…
- Produces Many **False-Positive Clusters**
- More Shading Work ☹

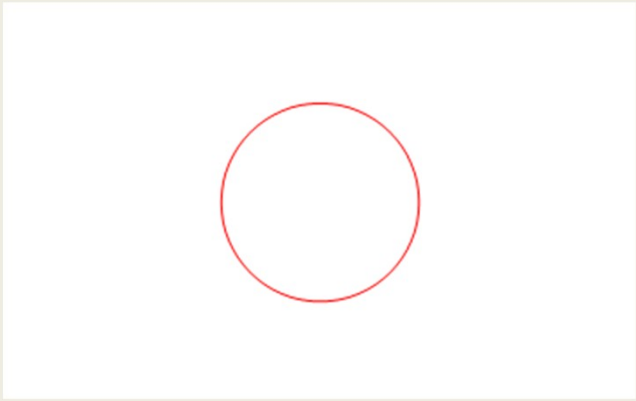
Solution: Cone vs Sphere Test

- Exact Mathematical Spot Light **Cone Volume**
- Cluster **Bounding Sphere**
- Perform Cone-Sphere Test

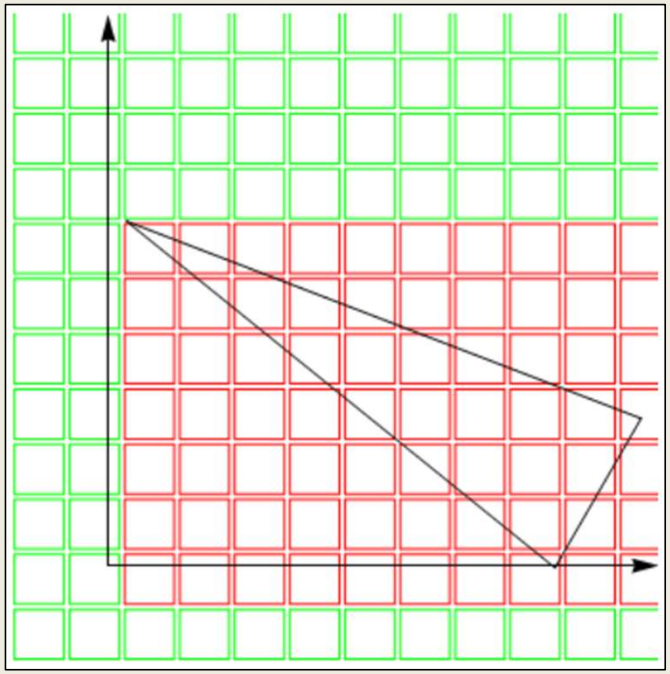
Trade-Off:

- „More Expensive" Intersection Test ☹
- A Lot Fewer Flase Positives!! ☺
- Lower Final Shading Cost!! ☺
- Worth the Extra Culling Cost

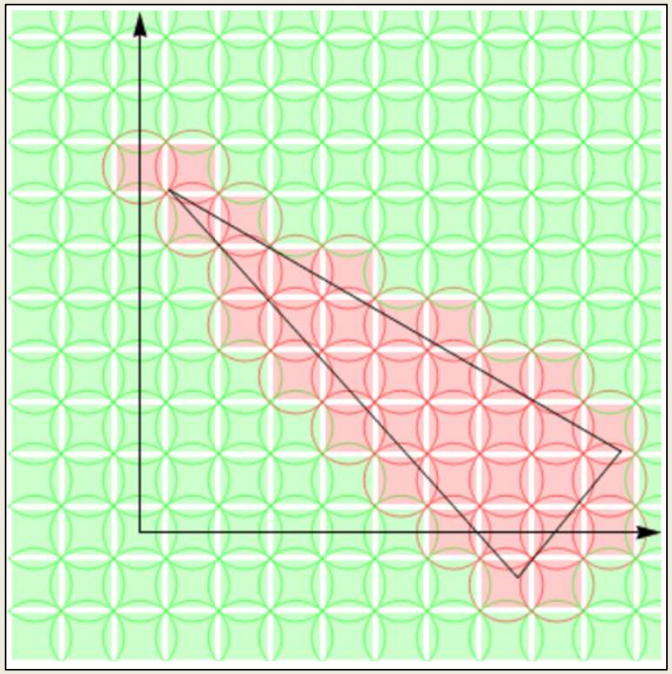
https://bartwronski.com/2017/04/13/cull-that-cone/



Spot Light Cone Sphere Collider



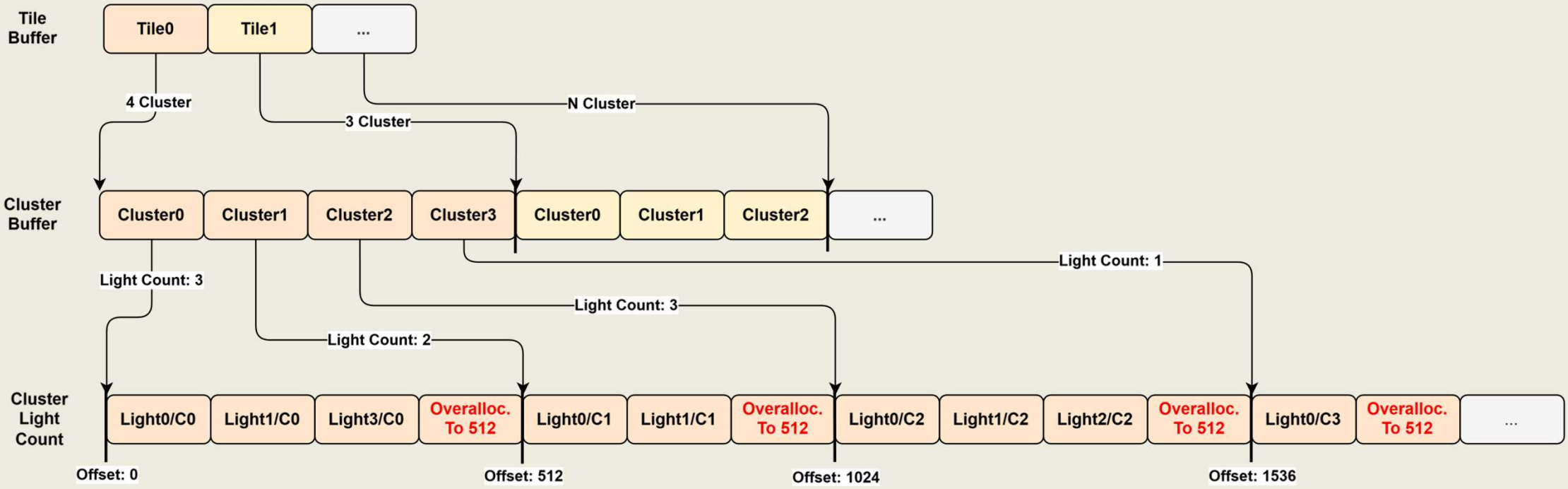
Cluster AABB vs Spot AABB



Spot Cone vs Cluster Sphere

Tile/Cluster/Light: Memory Layout

Fast Path (With Overallocation)!!!



Dual-HiZ Adaptive Clustered

Final Fragment Shader:

- Resolve the **XY Tile Index**
- Resolve the **Adaptive Cluster Index** from depth

Light Simulation

- Iterate only over the cluster's **Point Index Range**
- Iterate only over the cluster's **Spot Index Range**
- **Simulate only the lights assigned to that cluster!!!**

Reconstruct
World Position
View Depth

Resolve
Tile and Cluster Buffer Index!!

Light Simulation
Iterating over only lights in claster!!!

```
void main()
{
    // 1. Fetch Material Data

    uint cameraDenseIndex = GET_SPARSE_INDEX(ctx.cameraSparseMapBufferAddr, ctx.activeCameraEntity);
    CameraComponent camera = GET_CAMERA(ctx.cameraBufferAddr, cameraDenseIndex);

    vec2 screenUV = gl_FragCoord.xy / vec2(ctx.screenWidth, ctx.screenHeight);
    float fragDepth = gl_FragCoord.z;
    vec3 worldPos = ReconstructWorldPosition(screenUV, fragDepth, camera.viewProjVulkanInv);

    vec4 viewPos = camera.view * vec4(worldPos, 1.0);
    float viewDepth = abs(viewPos.z);
    vec3 viewDir = normalize(camera.eye.xyz - worldPos);

    uint tileX = uint(gl_FragCoord.x) / ctx.tileSize;
    uint tileY = uint(gl_FragCoord.y) / ctx.tileSize;
    uint tileIndex = tileY * ctx.tileCountX + tileX;

    TileData tile = GET_TILE_DATA(ctx.forwardPlusTileGridListBufferAddr, tileIndex);
    uint sliceIdx = GetClusterSliceIndex(viewDepth, tile.minZ, tile.maxZ, tile.sliceCount);

    uint clusterIndex = tile.clusterBaseOffset + sliceIdx;
    ClusterData cluster = GET_CLUSTER_DATA(ctx.forwardPlusClusterListBufferAddr, clusterIndex);

    // 7. Simulate Lighting
    vec3 totalRadiance = vec3(0.0);

    for(uint i = 0; i < ctx.activeDirectionLightCount; ++i) {
        vec3 lightContribution = SimulateDirectionalLight(...);
         totalRadiance += shadowFactor * lightContribution;
    }

    for (uint i = 0; i < cluster.pointLightCount; ++i) {
        vec3 lightContribution = SimulatePointLight(...);
        totalRadiance += shadowFactor * lightContribution;
    }

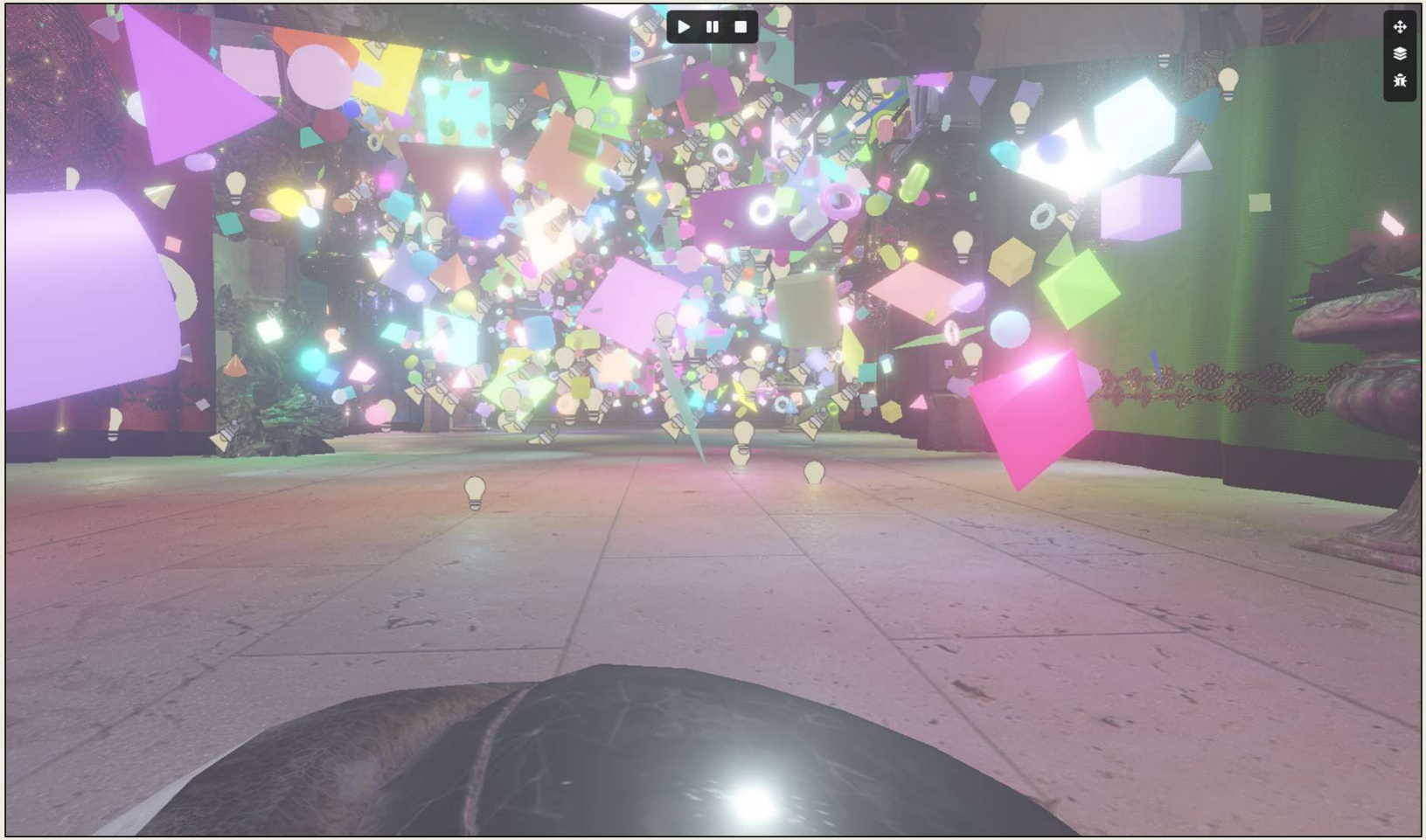
    for (uint i = 0; i < cluster.spotLightCount; ++i) {
        vec3 lightContribution = SimulateSpotLight(...);
        totalRadiance += shadowFactor * lightContribution;
    }

    outColor = vec4(totalRadiance, 1.0);
}
```

What if the scene contains 25.000 lights?

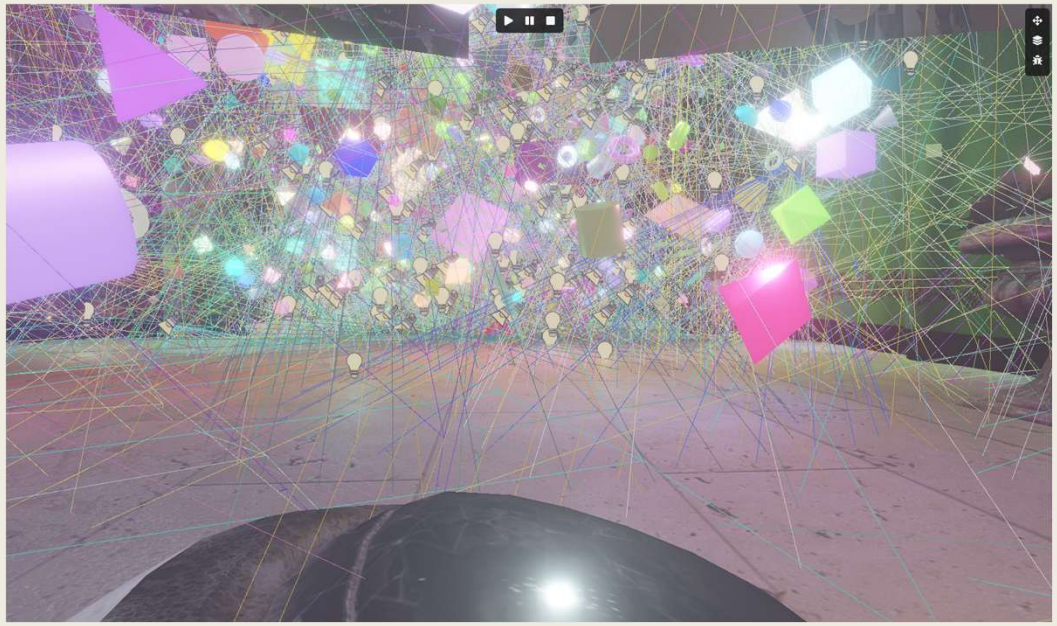
Such extreme cases are handled by the Scalable Path, presented at the end of the talk!

Forward+ Shading: Benchmark Scene

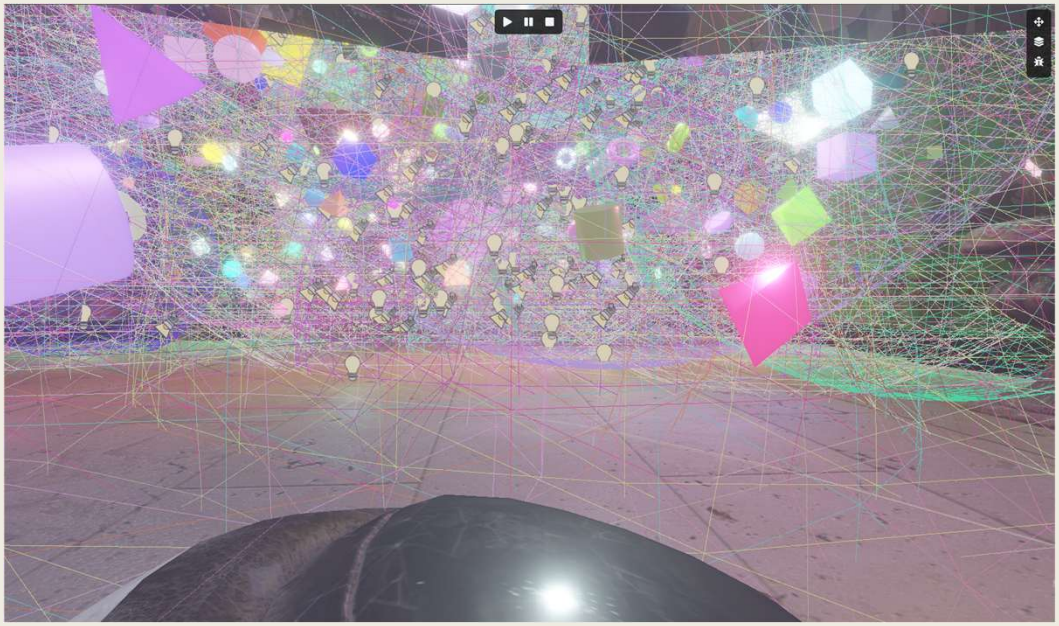


1 Sponza + 500k opaque shape + 1024 point + 1024 spot light randomly around

Forward+ Shading: Benchmark Scene

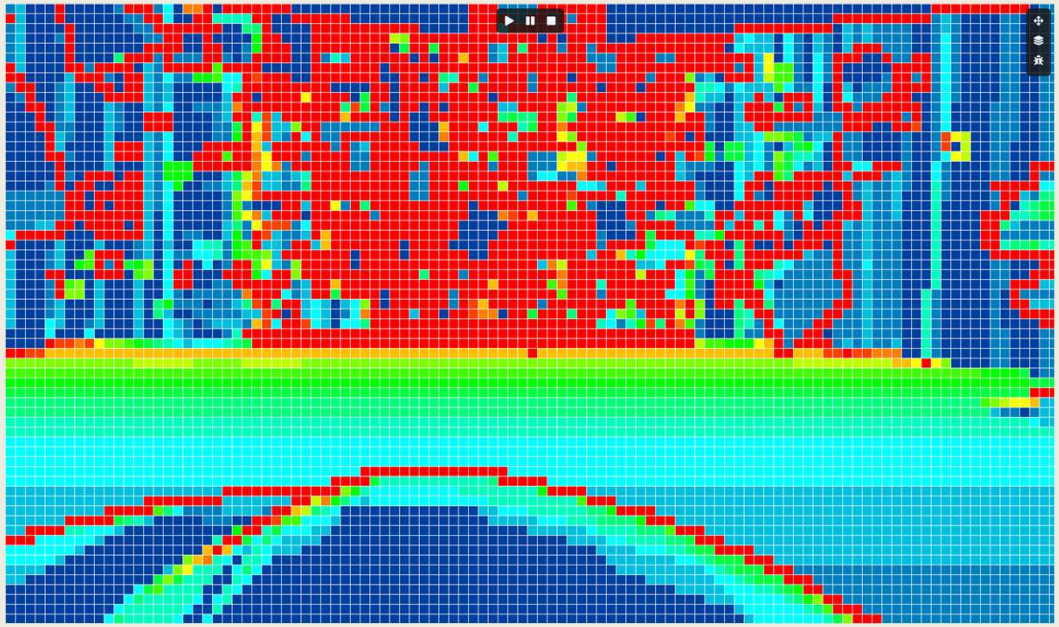


1024 Spot Light Volume Wireframe

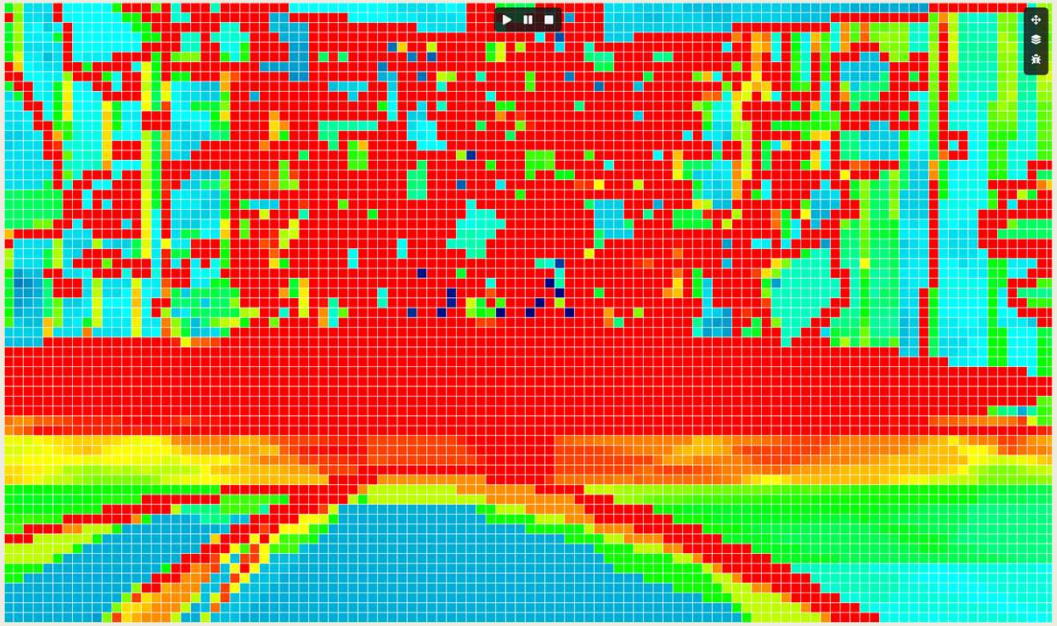


1024 Point Light Volume Wireframe

Forward+ Shading: Benchmark Scene

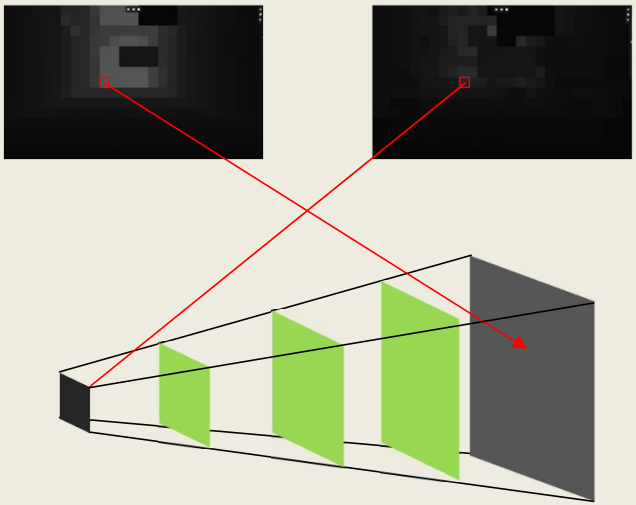


Dynamic Cluster Depth Heatmap
(DHAC)

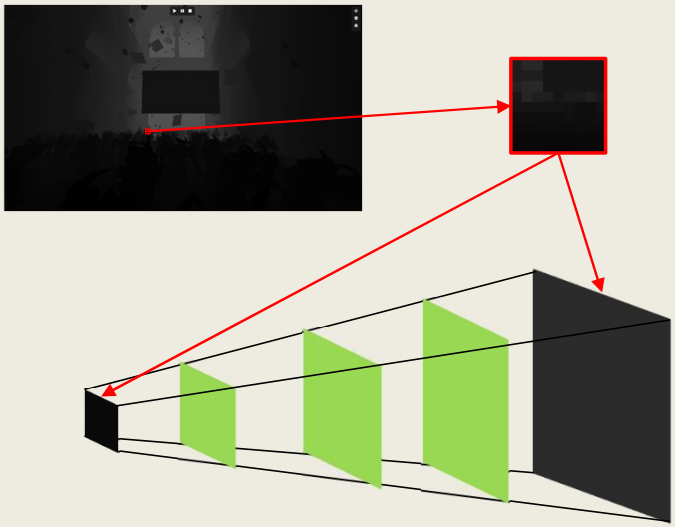


Cluster Light Source Heatmap
(DHAC)

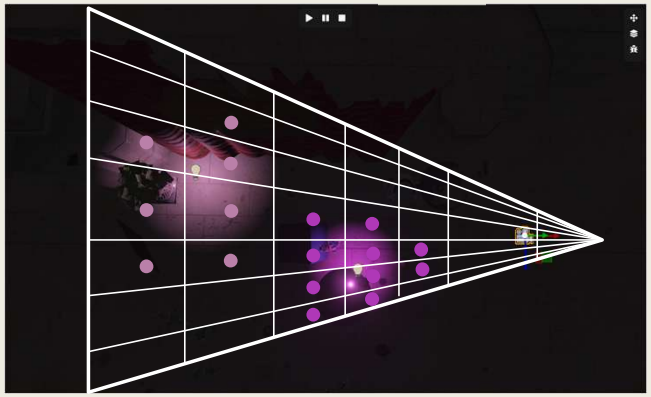
DHAC ↓



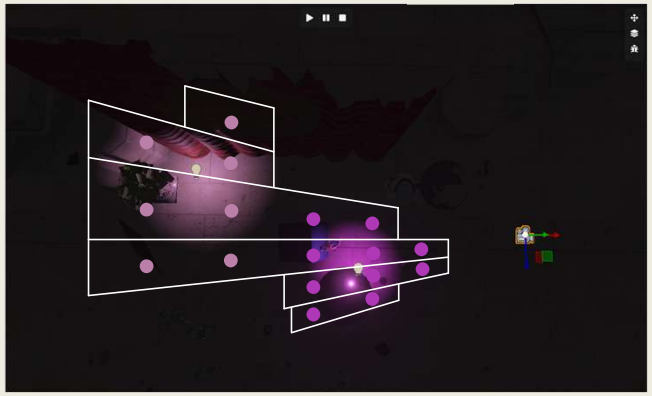
DHAC– ↓



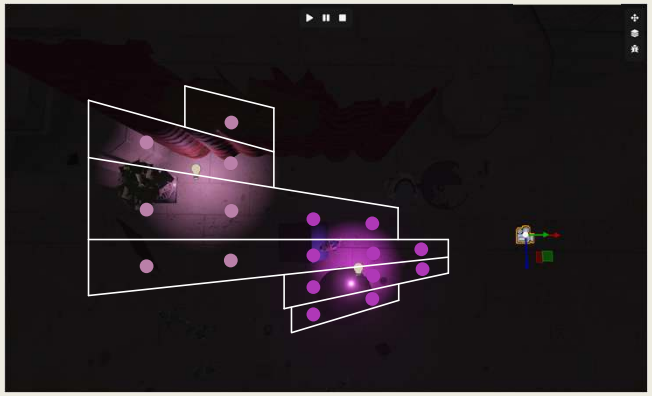
DOOM ↓



HARADA ↓



WICKED (Bitflag) ↓



Forward+ Shading:
Comprehensive Final Benchmarks

Algorithm	Configuration	Grid Size	Min Time (ms)	Speedup
DHAC	HiZ, 16 slices	32x32	3.083	1.00x
DHAC-	8x8 threads, 16 slices	32x32	3.375	1.09x
DOOM	32 slices	64x64	5.480	1.78x
WICKED	8x8 threads	8x8	6.923	2.25x
HARADA	16x16 threads	16x16	7.671	2.49x

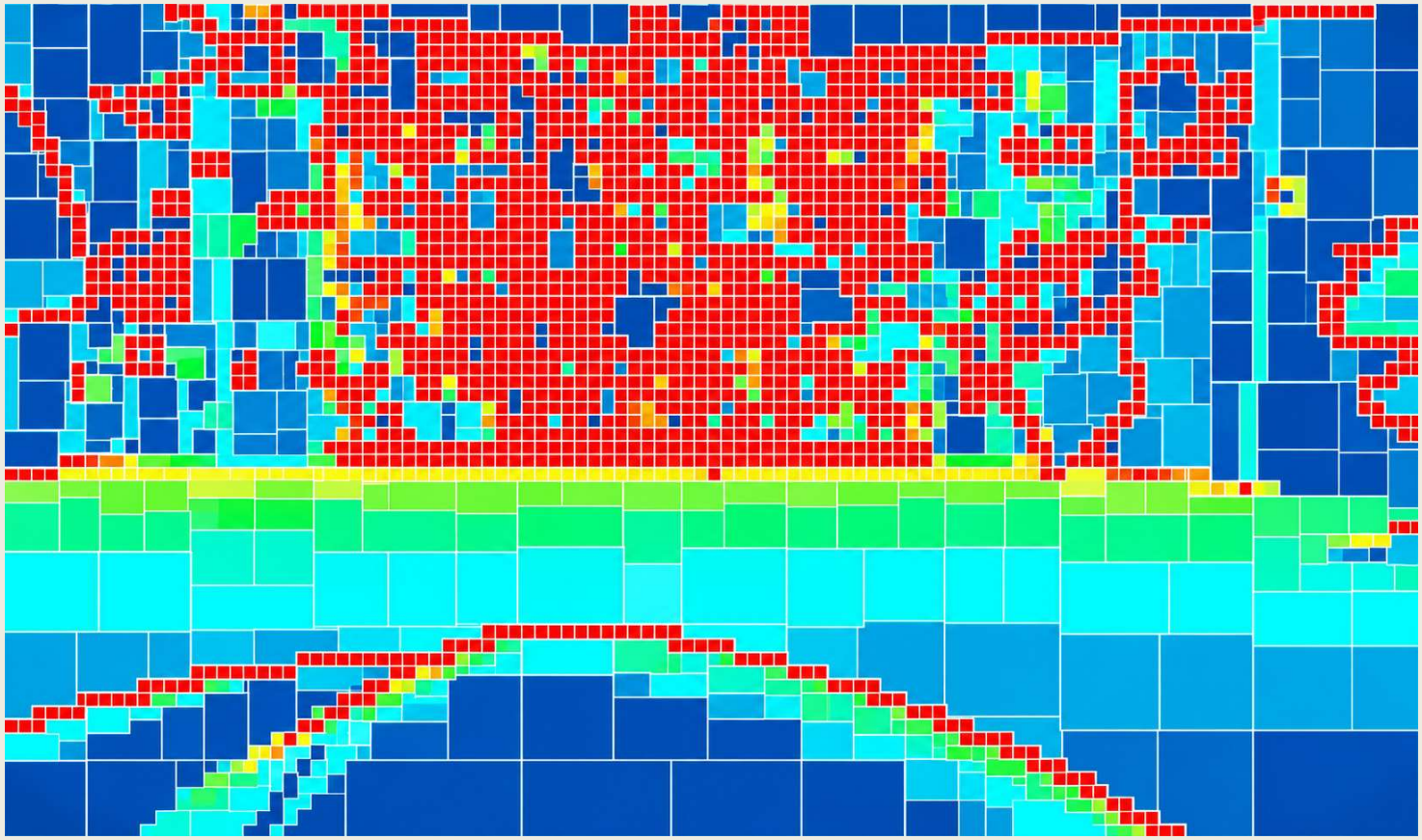
Table 3.10: Peak performance summary per algorithm family and the relative speedup of DHAC compared to their best configurations. (1024 point + 1024 spot light scene)

Forward+ Shading: Detailed Analysis

Technique	8x8	16x16	32x32	64x64	128x128
DHAC	7.801	4.213	**3.083**	3.578	3.657
DHAC- (8x8 threads, 16 slices)	8.581	4.281	**3.375**	3.716	4.338
DHAC- (16x16 threads, 16 slices)	-	3.947	3.760	**3.678**	4.625
DOOM (16 slices)	24.708	7.981	6.989	**6.733**	6.744
DOOM (16 slices, Frustum)	25.026	12.429	9.046	**8.806**	9.237
DOOM (32 slices)	43.077	10.901	5.971	**5.480**	6.732
DOOM (32 slices, Frustum)	54.265	15.467	9.511	8.781	**8.529**
HARADA (8x8 threads)	**7.936**	8.103	9.305	11.193	14.634
HARADA (8x8 threads, Frustum)	8.662	**7.685**	8.830	9.551	10.759
HARADA (16x16 threads)	-	**7.671**	8.926	10.719	13.576
HARADA (16x16 threads, Frustum)	-	**8.055**	8.257	9.375	10.293
WICKED (8x8 threads)	**6.923**	7.667	8.240	10.924	13.526
WICKED (8x8 threads, Frustum)	8.236	**7.859**	8.019	9.061	11.095
WICKED (16x16 threads)	-	**6.936**	8.053	9.993	12.828
WICKED (16x16 threads, Frustum)	-	8.020	**7.805**	8.685	10.763

Table 3.9: Comparison of rendering times for Forward+ light culling techniques (ms). Columns indicate screen-space tile dimensions in pixels. Bold values highlight the optimal resolution per configuration. (1024 point + 1024 spot light scene)

Future Directions I.



Dynamically Tiled Dual-Hiz Adaptive Clustered Forward+ (DTDHAC)

(AI-generated concept visualization. The feature is not implemented yet and is still under research.)

(For illustrative pruposes only!!!)

Future Directions II.
Visibility Buffer Shading

Already In Place

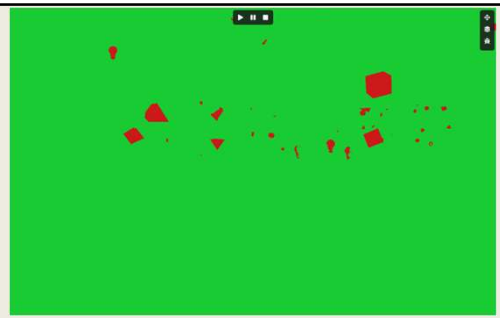
- **2x32 bit packed Visibility B.**
- R32 Word:
  1. Pipeline Type (1 bit)
  2. Entity ID (31 bit)
- G32 Word:
  1. Mesh ID (10 bit)
  2. LOD (2 bit)
  3. Meshlet ID (13 bit)
  4. Triangle Index (7 bit)]

Next Step

- Extend Visibility Data
- Reconstruct Triangle + Barycentrics
- Perform Lighting in Compute! ☺



← Entity ID



Pipeline Type →



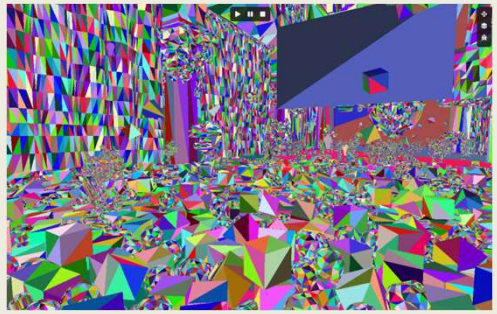
← LOD



Mesh Index→



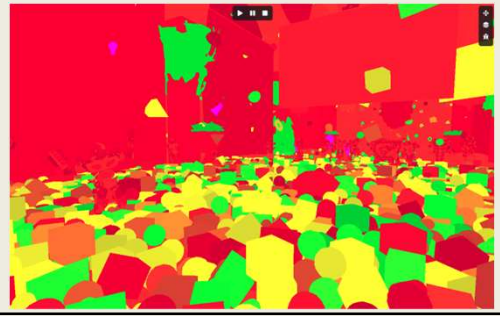
← Meshlet Index



Triangle Index→



← Pipelin/LOD/ Meshlet/ Combined



Material Type→

Questions?

Thank you for your attention!

WEBSITE:
HTTPS://SYNAPSEENGINE.DEV/
SOURCE CODE:
HTTPS://GITHUB.COM/TAMASPETII/SYNAPSEENGINE
BENCHMARKS:
HTTPS://SYNAPSEENGINE.DEV/DEVBLOG/02-FORWARD-PLUS-BENCHMARKS/
TECHNICAL PRESENTATION (250 PAGES):
HTTPS://SYNAPSEENGINE.DEV/PRESENTATIONS/SYNAPSEENGINEPRESENTATION.PDF

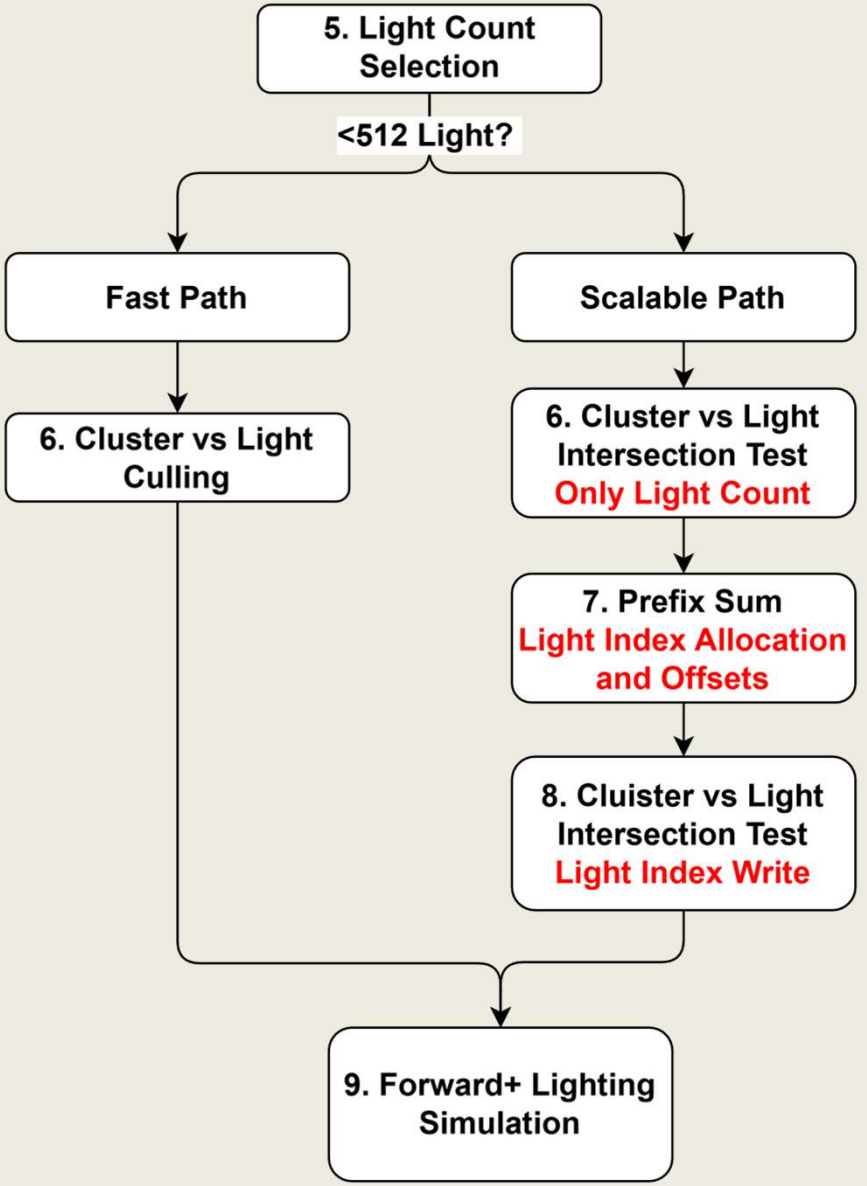
Dual-HiZ Adaptive Clustered Scalable Path

Problem with Allocation:

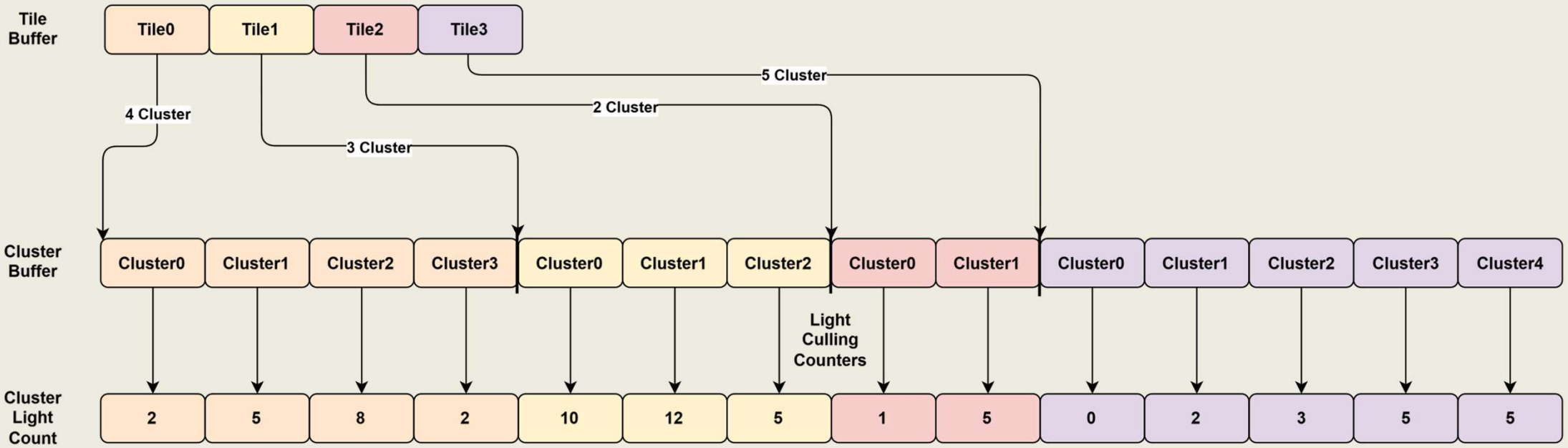
- **Fixed allocation does't scale to 1000s of lights**
- A single cluster may contain thousands of lights
- Worst-case preallocation wastes memory

Scalable Path

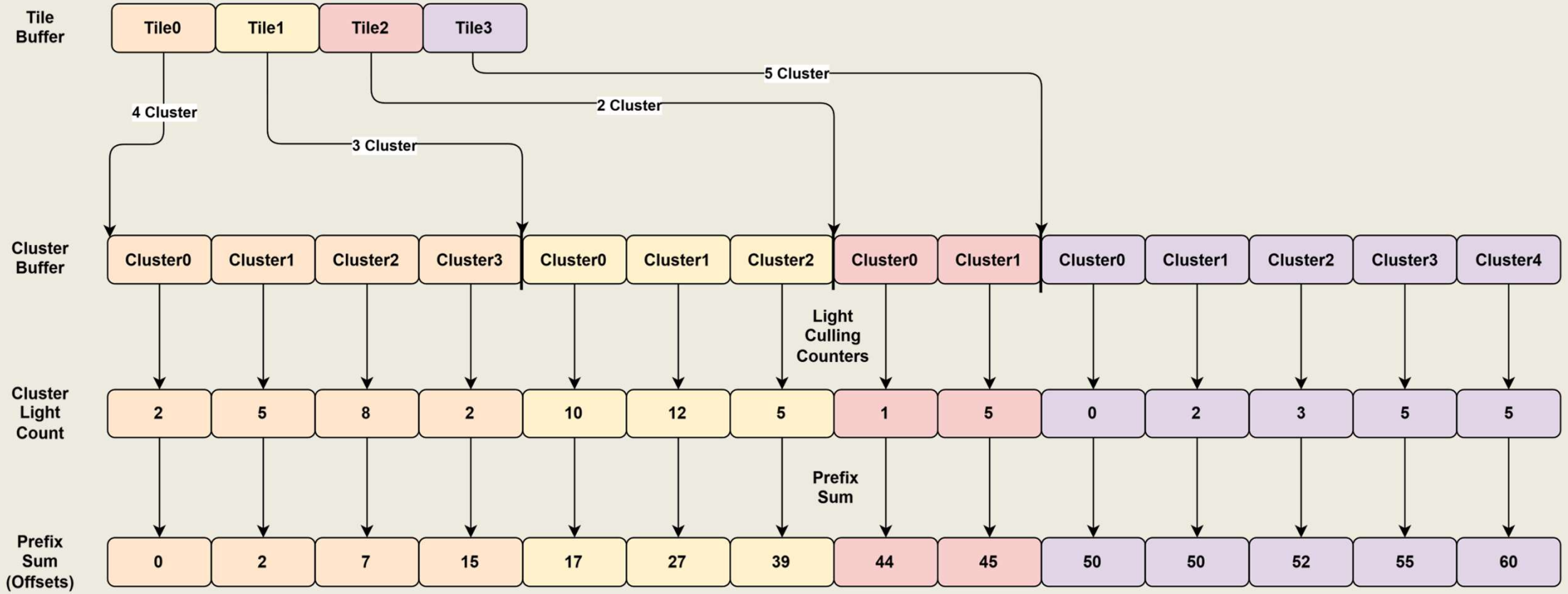
1. **Cluster Vs Light** intersection count
2. **Prefix sum**, compact offsets
3. **Cluster Vs Light** intersection write light indices to buffer



1. Cluster vs Light Count Pass



2. Parallel Prefix Sum Pass



Subgroup-Optimised Parallel Prefix Sum

1. Part of the Shader

```
// Single workgroup loops over all clusters. Size 512 is a safe maximum for all GPU vendors.
layout(local_size_x = 512, local_size_y = 1, local_size_z = 1) in;

shared uint s_pointSubgroupSums[64];
shared uint s_spotSubgroupSums[64];

shared uint s_pointSubgroupTotal;
shared uint s_spotSubgroupTotal;

// Carries the total sum from the previous 512-element block
shared uint s_globalPointCarry;
shared uint s_globalSpotCarry;

void main()
{
    // Loop through all active clusters in blocks of gl_WorkGroupSize.x (512)
    for (uint baseIdx = 0; baseIdx < totalClusters; baseIdx += gl_WorkGroupSize.x)
    {
        uint globalIdx = baseIdx + threadId;
        uint pCount = 0;
        uint sCount = 0;

        if (globalIdx < totalClusters) {
            ClusterData cluster = GET_CLUSTER_DATA(ctx.forwardPlusClusterListBufferAddr, globalIdx);
            pCount = cluster.pointLightCount;
            sCount = cluster.spotLightCount;
        }

        // 1. Hardware subgroup scan (Fast prefix sum within 32 threads)
        uint pSubOffset = subgroupExclusiveAdd(pCount);
        uint sSubOffset = subgroupExclusiveAdd(sCount);

        // 2. Total count within the subgroup
        uint pSubTotal = subgroupAdd(pCount);
        uint sSubTotal = subgroupAdd(sCount);

        // 3. Store subgroup totals in shared memory for cross-subgroup scanning
        if (subgroupElect()) {
            s_pointSubgroupSums[gl_SubgroupID] = pSubTotal;
            s_spotSubgroupSums[gl_SubgroupID]  = sSubTotal;
        }

        barrier();

        // ....
    }
}
```

2. Part of the Shader

```
void main()
{
    // Loop through all active clusters in blocks of gl_WorkGroupSize.x (512)
    for (uint baseIdx = 0; baseIdx < totalClusters; baseIdx += gl_WorkGroupSize.x)
    {
        // Previous Codes ...

        // 4. Thread 0 sequentially scans the subgroup totals
        if (threadId == 0)
        {
            uint pSum = 0;
            uint sSum = 0;

            for (uint i = 0; i < gl_NumSubgroups; ++i) {
                uint pVal = s_pointSubgroupSums[i];
                uint sVal = s_spotSubgroupSums[i];

                s_pointSubgroupSums[i] = pSum;
                s_spotSubgroupSums[i]  = sSum;

                pSum += pVal;
                sSum += sVal;
            }

            s_pointSubgroupTotal = pSum;
            s_spotSubgroupTotal  = sSum;
        }

        barrier();

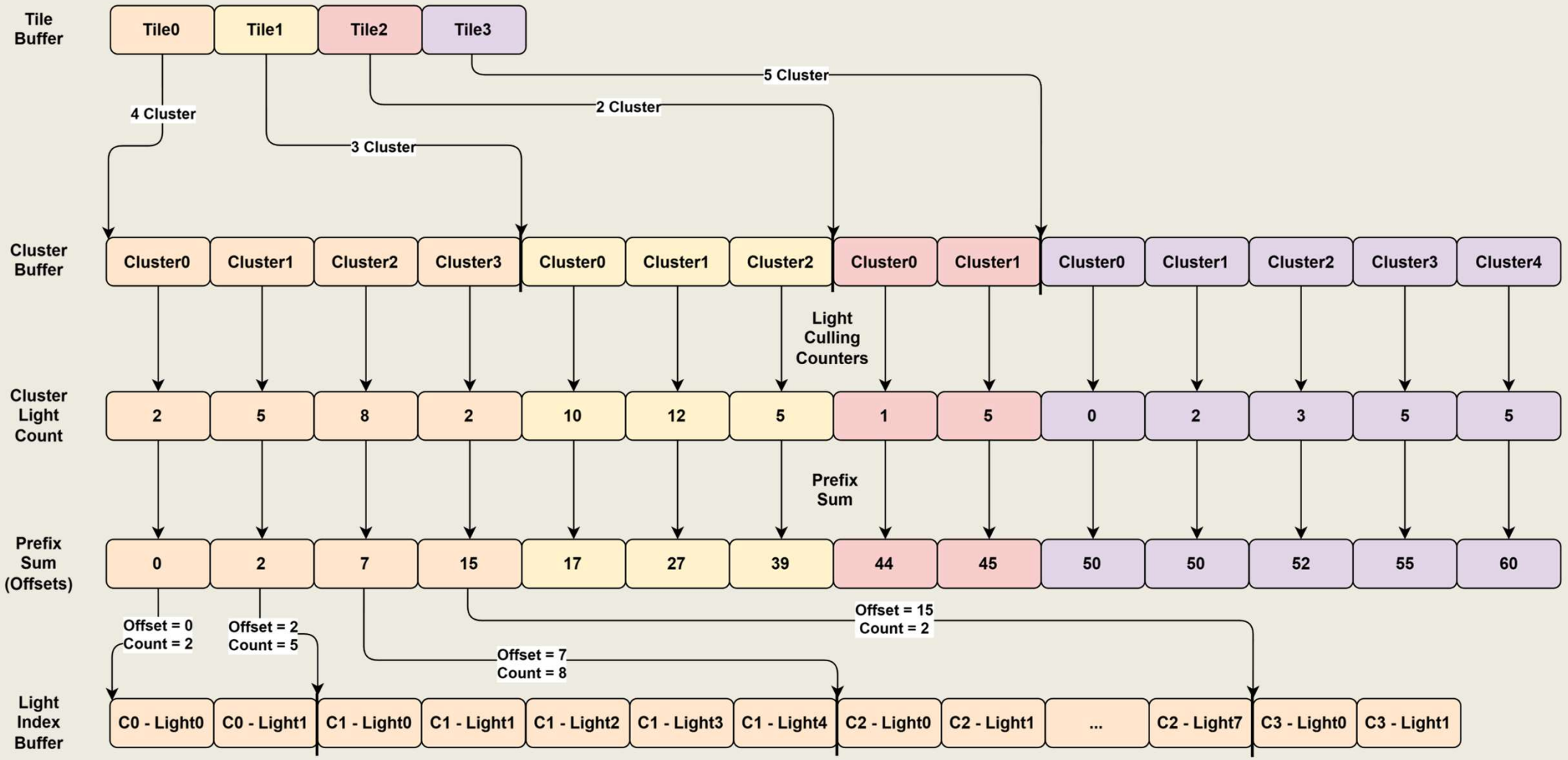
        // 5. Calculate final global offsets and write directly to VRAM
        if (globalIdx < totalClusters)
        {
            uint finalPOffset = s_globalPointCarry + s_pointSubgroupSums[gl_SubgroupID] + pSubOffset;
            uint finalSOffset = s_globalSpotCarry  + s_spotSubgroupSums[gl_SubgroupID] + sSubOffset;

            GET_CLUSTER_DATA(ctx.forwardPlusClusterListBufferAddr, globalIdx).pointLightOffset = finalPOffset;
            GET_CLUSTER_DATA(ctx.forwardPlusClusterListBufferAddr, globalIdx).spotLightOffset = finalSOffset;
        }

        // 6. Accumulate the carry for the next 512-element block
        if (threadId == 0) {
            s_globalPointCarry += s_pointSubgroupTotal;
            s_globalSpotCarry  += s_spotSubgroupTotal;
        }

        barrier();
    }
}
```

3. Cluster vs Light Write Pass



Dual-HiZ Adaptive Clustered Scalable Path

GPU-Driven Path Selection

- VK_EXT_conditional_rendering extension
- Select Fast / Scalable Path entirely **on the GPU**

Why do double Culling?

- **Trades additional ALU** for lower temporary memory / bandwith
- **Alternative:**
  1. Single Cull
  2. Temporary paris (Cluster | Light)
  3. Radix Sort
- Double culling avoids storing and sorting the intermediate intersection list! **(Not yet benchmarked)**

